Deep Learning

Elisa Ricci
a.y. 2025/2026
University of Trento

Davide Donà

June 2026

Notes Repository

Contents

1 Machine Learning Recap

Artificial Intelligence (AI) is the discipline that aims to create machines that mimic human intelligence. **Machine Learning** (ML) is a subset of AI that enables machines to learn from experience (i.e., data) to improve their performance on a specific task. **Deep Learning** is in turn a subfield of ML that focuses on **Artificial Neural Networks** (ANNs); the term 'deep' refers to the use of multiple layers, which allows the network to learn complex representations of data with multiple levels of abstraction.

Machine Learning was born in contrast to the programming paradigm, which is based on the idea of writing explicit rules to solve a problem.

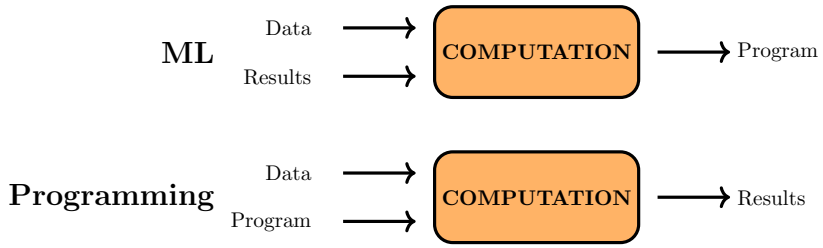


Figure 1: Programming paradigm vs Machine Learning paradigm.

In the Machine Learning paradigm, we are instead given the data, along with the results, and we try to learn the **model**. The learned model (e.g. splits in a decision tree, linear function for SVMs) represents the solution to the problem, and it can be used to make predictions on new data.

1.1 Why do we need Machine Learning

The main reason why we need Machine Learning can be summarized in the following points:

- **Complexity of the problem**: For many tasks, we have no human expertise to write explicit rules;
- **Personalized solutions**: For many tasks, we need to learn personalized solutions for each user, which is not feasible with the programming paradigm;
- **Huge amount of data**: Working with huge amount of data is not feasible, as it would require writing down explicit rules for each possible combination of inputs.

1.2 Machine Learning - Another definition

Machine Learning

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

This definition is more formal and precise than the previous one, and it highlights the importance of the three components: experience, task, and performance measure. Each machine learning algorithm can be characterized by these three components. For example, taking in consideration a spam email filter, we can define:

- T: Categorizing emails as spam or not spam;
- P: Accuracy of the classification. Other metrics may be more appropriate;
- E: Database of emails labeled as spam or not spam by human experts.

1.3 Types of learning

We can identify three main types of learning:

- **Supervised learning**: We are given a training set of input-output pairs, and we want to learn a function that maps inputs to outputs.
- **Unsupervised learning**: We are given a training set of inputs, and we want to learn a function that captures the underlying structure of the data.
- **Reinforcement learning**: We are given an environment, and we want to learn a policy that maximizes the cumulative reward.

1.3.1 Supervised Learning

Supervised learning can be further divided into subcategories, such as:

- **Classification**: The output is a discrete label. More formally, given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \{1, \ldots, k\} \tag{1.1}$$

  where $\mathcal{X}$ is the input space, and $\{1, \ldots, k\}$ is the set of possible labels.

  Based on the number of classes, we can further divide classification into **binary classification** and **multi-class classification**.

- **Regression**: The output is a continuous value. More formally, given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ we can define it as:

$$f : \mathcal{X} \to \mathbb{R} \tag{1.2}$$

- **Structured output**: **supervised learning** doesn't necessarily require a **vector** as input, and a **scalar** as output. We can have more complex input and output structures, such as images, graphs, or sequences. In all cases the output is a structured object, with relationships between its components. Some practical example of **structured input regression** problems are:

  - **Semantic segmentation**: Given an image of input size $M \times N \times 3$, we want to predict a mask of size $M \times N$, where each pixel is labeled with a class. The original solution to this kind of problem was to use **feature engineering** to extract features from neighborhoods of pixels. After that, SVMs were used to make predictions based on the extracted features. **Neural Networks** show their real power in this kind of problems, as they are able to learn features directly from the data, without any human intervention.
  - **Machine translation**: given a sentence (sequence of words) in a source language, we want to translate it to a target language. This is even more complex than the previous example, as not only the output is related to its adjacent components, but also grammatical and semantic relationships between words need to be captured.

1.3.2 Unsupervised Learning

In unsupervised learning, we are given a training set of inputs, without any corresponding output labels, and we want to learn the underlying structure of the data. Some examples of unsupervised learning tasks are:

- **Clustering**: grouping similar data points together;
- **Deep clustering**: a technique that combines deep learning with classical clustering algorithms, such as k-means, to learn better representations of the data and improve clustering performance. It is based on the idea of using a deep neural network to learn a low-dimensional representation of the data. This is then used as input to a clustering algorithm, producing a set of **pseudo-labels** that are used to train the neural network using backpropagation. This process is repeated iteratively until convergence;

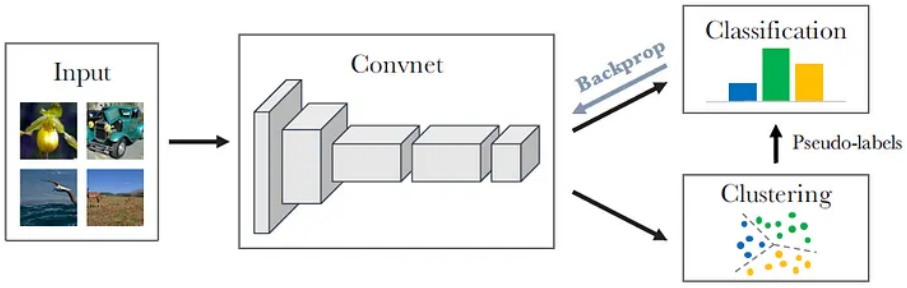


Figure 2: Deep clustering.

- **Generative Models**: models learn the underlying distribution of the data, and can be used to generate new data points, similar to the ones in the training set. Also in this case, we can have **deep generative models**. They try to map exaples extracted from a simple distribution Z to a more complex distribution $g_\theta(Z)$, which is similar to the distribution of X.

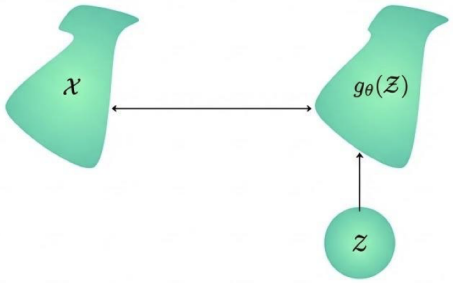


Figure 3: Deep generative models.

- **Dimensionality reduction**: try to map data to a lower-dimensional space in which visualization and interpretation are easier. This is often used as a preprocessing step for other machine learning algorithms.
- **Autoencoders**: a type of encoder-decoder architecture, where:
  - The **encoder** maps the input data to a **latent space** of lower dimensionality;
  - The **decoder** maps the latent representation back to the original input.

  The two components are trained together to minimize the reconstruction error, which is the difference between the original input and the reconstructed output. Once trained, the encoder can be used alone to obtain lower-dimensional representations of the data.

1.3.3 Reinforcement Learning

In Reinforcement Learning, an **agent** learn to interact with an **environment** in order to maximize a cumulative reward. We are not given any data, but rather the agent learns from its own experience, by exploring the environment and receiving feedback in the form of rewards.

In **deep reinforcement learning** the agent uses a deep neural network to learn a policy that maps states of the environment to actions. This allows the agent to learn complex behaviors and solve tasks that are difficult to solve with traditional reinforcement learning algorithms.

1.4 Designing a Learning System

To design a learning system, it's important to consider the following steps:

1. **Choose the training experience** E: This is the data that will be used to train the model. It should be representative of the problem we want to solve, and it should be of good quality;
2. **Choose the model**: This is the function that we want to learn from the data. It heavily depends on the task we want to solve,
3. **Choose how to represent the model**: the form of the model, what kind of function we allow. For example, we can have:
   - **Instance-based functions**: predictions are made based on the similarity between the input and the training examples. For example, k-nearest neighbors;
   - **Numerical functions**: predictions are made based on a mathematical function that maps inputs to outputs. For example, Neural Networks, SVMs;
   - **Probabilistic functions**: predictions are made based on a probability distribution over the possible outputs. For example, Naive Bayes;
   - **Symbolic functions**: predictions are made based on a set of rules that are learned from the data. For example, decision trees.
4. **Choose the learning algorithm**: This is the algorithm that will be used to learn the model from the data. For example, for SVMs we can decide between different optimization algorithms, such as stochastic gradient descent or quadratic optimization.

   The success of the machine learning system also depends on the search and optimization algorithm used to find the best model.

5. **Choose the performance measure** P: This is the metric that will be used to evaluate the performance of the model. Different tasks require different performance measures and it's not always a trivial choice. Ideally we would like to choose a performance measure that is closely related to the loss function used to train the model.

1.5 Assumptions in Machine Learning

When developing a machine learning model, we usually assume that we are given a training set of examples that are **observed** and we want to learn a model that can make predictions on new, unseen examples, the **test set**.

The challenge is that the algorithm has to learn a model that performs well on new examples. This ability is known as **generalization**.

1.5.1 Empirical risk minimization

Classical optimization algorithms are different from machine learning algorithms. In classical optimization, the only goal is to find the best solution to a problem, reducing the **training error** as much as possible. In machine learning instead, we want to minimize the **test error**, which is the error on new, unseen examples. This is a much harder problem, as we don't have direct access to the test set during training.

To tackle this problem, we assume that both training and test examples are drawn from the same distribution. This is known as the **independent and identically distributed** (i.i.d.) assumption, and it allows us to use the training set to learn a model that generalizes well to the test set.

If the training set is representative of the underlying unknown distribution, then the **empirical loss** (the average loss on the training set) is a good estimate of the **expected loss** (the average loss on the test set). This is the principle of **empirical risk minimization**.

1.5.2 Assumption violations

If the training set is not representative of the underlying distribution or if the i.i.d. assumption is violated, then the model may not generalize well to the test set, and we may end up with a model that performs well on the training set but poorly on the test set. This is known as **overfitting**.

1.5.3 Underfitting

If the model is too simple, it may perform poorly on both the training and test set. This is known as **underfitting**. To avoid underfitting, we need to choose

a more complex model, or to use a more powerful learning algorithm.

1.5.4 Overfitting

If the model is too complex, it may perform well on the training set but poorly on the test set. When the gap between the two errors is large, it means that the model has learned to fit the noise in the training data, rather than the underlying distribution. This is known as **overfitting**.

To avoid overfitting, we can use techniques such as:

- **Simpler models**: choosing a simpler model with fewer parameters, which is less likely to overfit the training data;
- **Regularization**: any modification made to a learning algorithm that is intended to reduce the generalization error, but not the training error. Usually, this is done by adding a penalty term to the loss function, which is proportional to the complexity of the model (e.g., adding a term that penalizes large weights in a neural network).

1.6 How to choose the algorithm

To choose the right algorithm for a given problem, we need to consider the following factors:

- **Size of the training set**: Some algorithms perform better with small training sets, while others require large training sets to perform well;
- **Address problem difficulty**: are data linearly separable? Is the problem a regression or a classification problem? Is the output structured?
- **Prior knowledge**: if prior knowledge about the problem is available, it can be encoded in the model, which can improve performance;
- **Computational requirements**: do we have any requirements on the computational resources available for training and inference?
- **Online vs batch learning**: based on the kind of data we have, we may need to choose different algorithms. For example, for streaming data, Neural Network aren't the best choice;
- **Interpretability**: do we need to understand how the model makes predictions?

2 Feedforward Neural Networks

Before we can understand the architecture of a **feedforward neural network** (FFNN), we first introduce how we got here.

2.1 From the Perceptron to Deep Networks

Before introducing the architecture of a feedforward neural network, it is useful to recall the building block from which it originated: the artificial neuron.

Electronic Brain In the first perceptron model, the inputs form a vector of **binary features** that are simply summed together and passed through a **threshold function**, without any **weights** involved.

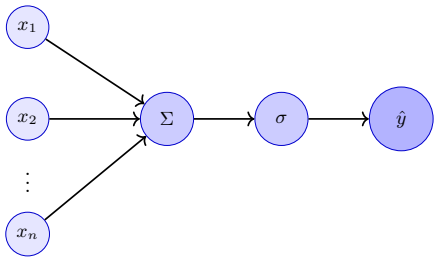


Figure 4: Architecture of the electronic brain model. Inputs x_i are summed together and passed through an activation function σ to produce the output $\hat{y}$.

It can be shown that this model is capable of classifying linearly separable data, such as the AND, OR, and NOT functions, but it fails to classify non-linearly separable data, such as the XOR function.

The Perceptron The **Perceptron** is an extension of the electronic brain model. The key difference is the introduction of **weights** for each input, which allows the model to learn from data and adjust its parameters accordingly.

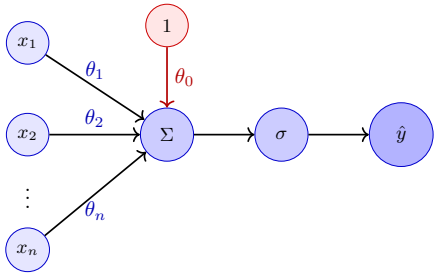


Figure 5: Architecture of a single layer perceptron. Inputs x_i are weighted by w_i, summed together, and passed through an activation function σ to produce the output y.

The prediction $\hat{y}$ of the perceptron is given by the following formula:

$$\hat{y} = \sigma\left(\sum_{i=1}^{n} \theta_i x_i + \theta_0\right) \tag{2.1}$$

where θ_i are the weights associated with each input x_i, and σ is the activation function. We can also express the prediction in vector form as follows:

$$\hat{y} = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x} + \theta_0\right) \tag{2.2}$$

The most important intuition is that the weights θ_i are not fixed, but rather learned from data.

The bias term is introduced to allow the model to shift the decision boundary (hyperplane) away from the origin.

Perceptron

An artificial neuron, i.e., a perceptron is a non-linear parametrized function with a restricted output range.

Today, the activation function σ is typically a non-linear function, such as the sigmoid, ReLU, or tanh function, which allows the model to learn complex patterns in the data.

Perceptron Learning Algorithm The weights of the perceptron are learned from data using the **Perceptron Training Rule**. Given a set of training examples $\mathcal{T} = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots, (\boldsymbol{x_N}, y_N)\}$, $y_i \in \{0, 1\}$ and a learning rate η, the weights are updated as follows:

1. Initialize the weights θ_i to small random values;

2. Until convergence, repeat:
   (a) Select randomly a training example $(\boldsymbol{x_i}, y_i)$ from $\mathcal{T}$;
   (b) Compute the prediction $\hat{y}_i$ using the current weights:
   $$\hat{y}_i = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x_i} + \theta_0\right)$$
   (c) Update the weights based on the error between the prediction and the true label:
   $$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} + \eta(y_i - \hat{y}_i)\boldsymbol{x_i}$$
   (d) Update the bias:
   $$\theta_0^{(t+1)} \leftarrow \theta_0^{(t)} + \eta(y_i - \hat{y}_i)$$

Note that we are assuming that the training data is linearly separable, otherwise we would have to introduce a stopping criterion to prevent the algorithm from running indefinitely.

MLP A single perceptron cannot learn non-linearly separable data, such as the XOR function. This limitation motivated the **Multilayer Perceptron** (MLP), which stacks several layers of neurons.

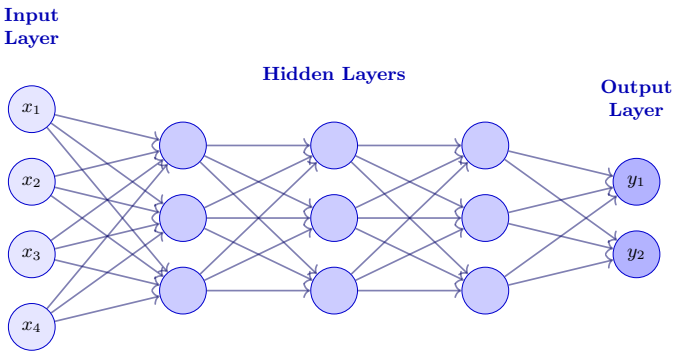


Figure 6: Architecture of a Multi-Layer Perceptron (MLP) with three hidden layers, fully connected in a feed-forward fashion.

This structure is very flexible: multiple nodes can be used in the output layer, so it can be applied to multi-class classification problems, and also to regression.

Backpropagation The perceptron learning algorithm is not suitable for training MLPs, as no expected output is available for the hidden layers, but just for the output layer.

Backpropagation is an efficient algorithm for computing the gradients also for large datasets. At a high level, the algorithm can be summarized as follows:

1. **Forward propagation**: Given an input $\boldsymbol{x}$, compute the output of the network $\hat{y}$ by propagating the input through the layers of the network;
2. **Loss computation**: Compute the loss $L(y, f(\boldsymbol{x}, \boldsymbol{\theta}))$, where y is the true label and $f(\boldsymbol{x}, \boldsymbol{\theta})$ is the output of the network with parameters $\boldsymbol{\theta}$;
3. **Backward propagation**: the error signal is propagated backwards through the network and it's used to update the weights.

This concept is covered in more detail later in this chapter.

Cybenko's Theorem Cybenko's theorem states that a 2-layer feed-forward neural network with linear output can approximate any continuous function on a compact subset of $\mathbb{R}^n$ with arbitrary accuracy.

In practice, the more hidden layers we have, the more complex functions we can learn. Deep neural networks need a lot of labeled data to be trained, and cannot be trained easily.

2.2 From Logistic Regression to Feedforward NN

To fit a model to some data distribution, a simple and common approach is to use **Maximum Likelihood Estimation** (MLE).

2.2.1 Maximum Likelihood Estimation

In MLE, we want to estimate the parameters of an assumed probability distribution, given some observed data. For example, if we assume that the data is generated from a Gaussian distribution, we can estimate the mean and variance of that distribution using MLE.

More formally, given a dataset $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ and a known probability distribution $p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})$ we want to find the parameters $\boldsymbol{\theta}$ that better explain the data.

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) \tag{2.3}$$

Assuming that the data points are independent and identically distributed (i.i.d.), we can express the likelihood as a product of individual probabilities:

$$p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \prod_{i=1}^{N} p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

To simplify the optimization process, we often take the logarithm of the likelihood function, which transforms the product into a sum:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p_{\text{model}}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

We can show that this is equivalent to considering the expected value of the log-likelihood over the empirical (observed) data distribution:

$$\log p_{\text{model}}(\boldsymbol{X}|\boldsymbol{\theta}) = N \cdot \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}[\log p_{\text{model}}(\boldsymbol{x}|\boldsymbol{\theta})]$$

where p_{data} is the empirical distribution of the observed data and $\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}$ denotes the expected value sampled from this distribution.

To find the optimal parameters $\boldsymbol{\theta}^*$, we usually differentiate the log-likelihood with respect to $\boldsymbol{\theta}$ and set it to zero. We can then resolve the resulting equation to find the optimal parameters.

The equation 2.3 can also be extended to the case of conditional probability distributions ($p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$) in order to predict a target variable y given an input variable $\boldsymbol{x}$:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} p_{\text{model}}(y|\boldsymbol{x}, \boldsymbol{\theta})$$

This is the basis for most supervised learning algorithms, including Linear Regression and Logistic Regression, which simply assume different forms for the conditional probability distribution. For example, in Logistic Regression, we assume that the conditional probability distribution is given by the logistic function:

$$p_{\text{model}}(y = 1|\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T\boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T\boldsymbol{x}}} \tag{2.4}$$

2.2.2 Logistic Regression

Logistic Regression is a linear model used for binary classification tasks. Given the equation 2.4, we can observe that the can be seen as a simple Neural Network with a single layer and a sigmoid activation function.

2.3 Feedforward Neural Networks

A Feedforward Neural Network (FFNN) is a type of artificial neural network that define a mapping from the input space to an output space. The learned mapping $f(\boldsymbol{x})$ is defined as a composition of several functions, each representing a layer of the network.

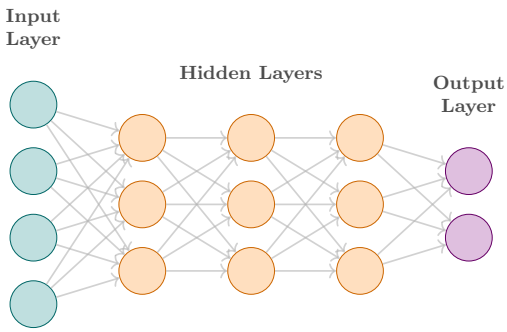


Figure 7: Example of a feedforward neural network with three hidden layers.

For example, taking in consideration the FFNN architecture shown in the figure 7 with three layers, we can express the mapping as follows:

$$f(\boldsymbol{x}) = f^{(5)}(f^{(4)}(f^{(3)}(f^{(2)}(f^{(1)}(\boldsymbol{x})))))$$

- The first layer $f^{(1)}(\boldsymbol{x})$ is usually referred to as the **input layer**, each node corresponds to a feature of the input data. Usually no computations are performed in this layer, as it simply serves as a conduit for the input data to be fed into the network;
- The layers $f^{(2)}$, $f^{(3)}$, and $f^{(4)}$ are called **hidden layers**, and they perform computations on the input data, applying a linear transformation followed by a non-linear activation function. The number of hidden layers and the number of neurons in each hidden layer depend on the architecture of the network and the complexity of the problem being solved.
- The last layer $f^{(5)}$ is called the **output layer**, and it produces the final output of the network. Depending on the type of problem, different activation functions can be used (e.g., linear for regression, softmax for classification) and different number of neurons can be used (e.g., one for binary classification and regression, one per class for multi-class classification).

The function composition can be described by a **directed acyclic graph** (DAG), where the nodes represent the functions (layers) and the edges represent the flow of data through the network. The term "feedforward" comes from the fact that the data flows in one direction, from the input layer to the output layer, without any cycles or loops in the graph.

2.3.1 Difference from Linear Models

Linear models are generally very easy to optimize (e.g., closed-form solutions or convex optimization), but they are limited in their expressiveness and cannot

capture interactions between features or non-linear relationships in the data. FFNNs allow to overcome the limitations of linear models by introducing non-linear activation functions and multiple layers of computation.

2.3.2 How can we improve Linear Models?

FFNNs are not the only way to improve linear models, but they are one of the most powerful and widely used approaches. Generally, to boost the performance of linear models, we can apply a non-linear transformation to the input data $\boldsymbol{x} \rightarrow \phi(\boldsymbol{x})$, where ϕ is a non-linear function. We can then apply a linear model to the transformed data $\phi(\boldsymbol{x})$. The transformation ϕ can be implemented in various ways, such as:

- **Kernel Methods**: map the input data into a high-dimensional feature space (possibly infinite-dimensional) where linear models can be applied. This has enough capacity to capture complex relationships in the data, but achieves poor generalization performance for highly variable target functions.
- **Feature Engineering**: manually design and select features that capture relevant information from the input data. This was the traditional approach before the rise of deep learning, in particular for computer vision tasks, but it is time-consuming and requires domain expertise.
- **Neural Networks**: learn the transformation ϕ from the data itself. This shifts the engineering effort from designing features to designing the architecture of the network, but throwhs away the convexity and interpretability.

2.3.3 Training a FFNN

To train a Neural Network, we need to optimize the parameters of the network (weights $\boldsymbol{\theta}$) to drive the output of the network $f(\boldsymbol{x}; \boldsymbol{\theta})$ to match the target variable y for a given input $\boldsymbol{x}$.

The training process is nothing different from training any other machine learning model. We need to define a loss function $\mathcal{L}(f(\boldsymbol{x}; \boldsymbol{\theta}), y)$ that measures the discrepancy between the predicted output and the true target variable.

The largest difference with respect to linear models comes from the fact that most loss functions are non-convex, which removes the guarantee of finding a global minimum and makes the optimization process more challenging.

2.3.4 Modelling Choices

When designing a FFNN, we need to make several modelling choices, such as:

- **Cost Function**: the choice of the loss function $\mathcal{L}$ depends on the type of problem we are trying to solve (e.g., regression, classification) and the properties of the data. Each function has its own advantages and disadvantages, and the choice can significantly impact the performance of the model.
- **Form of Output**: the function applied in the output layer. Similarly to the cost function, the choice of the output units depends on the type of problem we are trying to solve;
- **Activation Function**: the non-linear function applied in the hidden layers. It's independent from the type of problem we are trying to solve, but it can significantly impact the performance of the model and the convergence during training.
- **Architecture**:
- **Optimizer**:
- **Regularizer**:

2.4 Cost Functions

The porpouse of cost functions is to measure the discrepancy between the predicted output of the network and the true target variable, guiding the parameter optimization process. Various functions can be defined based on the type of problem.

Classification

- **Categorical Cross-Entropy Loss**: used for multi-class classification problems. It measures the dissimilarity between the predicted probability of class $\hat{y}_i$ and the true class label. For a single data point, the loss can be expressed as:

$$L(y, \hat{y}) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

- **Binary Cross-Entropy Loss**: special case of the categorical cross-entropy loss for binary classification problems. For a single data point, the loss can be expressed as:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Regression

- **Mean Squared Error (MSE)**: commonly used for regression problems. It measures the average squared difference between the predicted and true values. For a single data point, the loss can be expressed as:

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

  It penalizes larger errors more than smaller ones, as the difference is squared. This also makes the loss function convex, which is easier to optimize, but it can be very sensitive to outliers in the data, as they can have a disproportionate influence on the overall loss.

- **Mean Absolute Error (MAE)**: similar to MSE but uses the absolute difference instead of the squared difference. For a single data point, the loss can be expressed as:

$$L(y, \hat{y}) = |y - \hat{y}|$$

  This one penalizes large errors less than MSE, making it more robust to outliers in the data.

- **Huber Loss**: combines the properties of MSE and MAE. It behaves like MSE for small errors and like MAE for large errors, providing a balance between sensitivity to outliers and smoothness of the loss function. Formally, for a single data point, the loss can be expressed as:

$$L(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{for } |y - \hat{y}| > \delta \end{cases}$$

  where δ is a hyperparameter that determines the threshold between the MSE and MAE behavior.

Many more loss functions exist, and can be tryed out based on the specific problem and data characteristics.

Contrastive Learning

Another branch that is worth mentioning is the one used to train models in a self-supervised way, such as contrastive learning. As shown in the figure 8, the idea is to learn a representation of the data that captures the underlying structure and relationships between data points, that is invariant to certain transformations.

The loss function used in contrastive learning is designed to encourage the model to learn representations that are similar for similar data points and dissimilar

for different data points. A possible loss function for contrastive learning is the **Noise Contrastive Estimation** (NCE) loss, which can be expressed as follows:

$$NCE_{Loss} = -\log \frac{e^{\text{sim}(\boldsymbol{z_i}, \boldsymbol{z_j})}}{e^{\text{sim}(\boldsymbol{z_i}, \boldsymbol{z_j})} + \sum_{k=1}^{N} e^{\text{sim}(\boldsymbol{z_i}, \boldsymbol{z_k})}}$$

where $\boldsymbol{z_i}$ and $\boldsymbol{z_j}$ are the representations of two similar data points (anchor and positive), $\boldsymbol{z_k}$ are the representations of all data points (negatives), $\text{sim}(\cdot, \cdot)$ is a similarity function (e.g., cosine similarity).

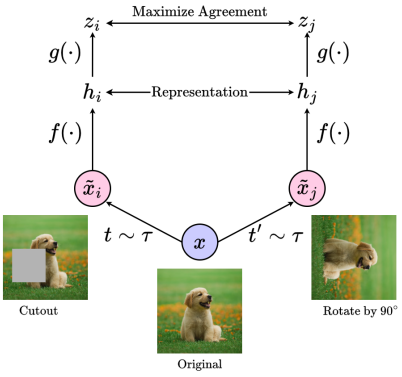


Figure 8: Contrastive learning Model

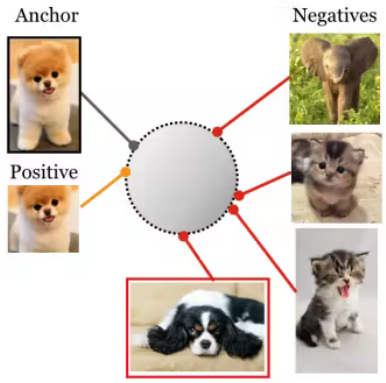


Figure 9: Contrastive learning loss function

2.5 Output Units

The output units are the last layer of the network, and their choice depends on the type of problem we are trying to solve.

- **Linear Units**: used for regression problems, where the output is simply a linear combination of the inputs.

$$y = \boldsymbol{\theta}^T \boldsymbol{x} + \theta_0$$

- **Softmax Units**: used for multi-class classification problems. They allow to take the unnormalized output of the last hidden layer and convert it into a probability distribution over the classes. For a given input $\boldsymbol{x}$, the output of the softmax layer can be expressed as:

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}$$

where z_i is the output of the last hidden layer for class i, and K is the total number of classes.

This produces a valid probability distribution over the classes, which also handles the multi-class classification case in a natural way.

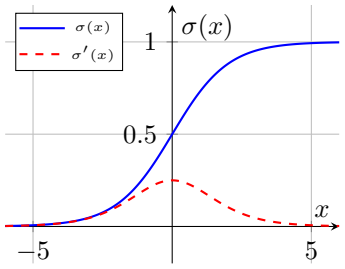
2.6 Activation Functions

Hidden units usually use a different activation function than the output units as it is independent from the type of problem we are trying to solve.

What's important to note is that in the choice of activation function it is much more important to consider the derivative of the function rather than the function itself, as it is what will be used during the backpropagation process to update the weights of the network.

Sigmoid Maps the input to a value between 0 and 1, which can be interpreted as a probability. It is defined as:

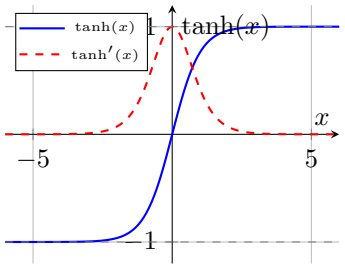
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



This function is smooth and differentiable, but it saturates (approaches 0) across most of its domain, which can lead to the problem of vanishing gradients during training.

Hyperbolic Tangent (tanh) Similar to sigmoid but maps the input to a value between -1 and 1. It is defined as:

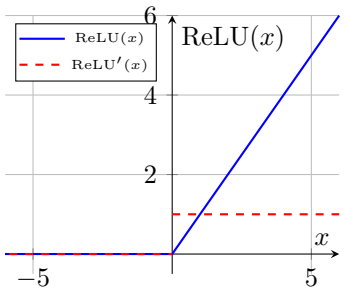
$$\tanh(x) = 2\sigma(2x) - 1 = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



This is better than the sigmoid function as it is zero-centered, which can help with convergence during training, but it still suffers from the saturation problem for large positive and negative values.

ReLU The most commonly used activation function in hidden layers. It is defined as:

$$\text{ReLU}(x) = \max(0, x)$$



Born to solve the saturation problem of sigmoid and tanh, the ReLU function is in fact non-saturating for positive values (derivative is 1) which allows for faster convergence during training.

However, this function has still a few negative aspects:

- **Non zero centered output**: similarly to the sigmoid, also the output of the ReLU is non-zero centered;
- **Dying ReLU**: Whenever the input of a neuron is negative, the output is zero, leading to fewer neurons being activated. This leads to the "dying ReLU" problem, where neurons can become inactive and only output zero for all inputs, which can hinder learning. This usually happens when most of the inputs for a neuron are in the negative range, making the function return 0. Also, the gradients for these neurons will be zero, which means that during backpropagation, the weights of these neurons will not be updated, leading to a situation where the neuron is "dead" and cannot recover.

Different solutions have been proposed to solve the dying ReLU problem, such as:

- **Stochastic Gradient Descent (SGD)**: the randomness introduced by SGD can help to prevent neurons from dying, as it allows for some updates to the weights even for neurons that are currently inactive.
- **Lower Learning Rates**: using too high learning rates can cause the weights to update too aggressively, possibly leading to weights in the highly negative range;

- **Generalized ReLU**: A family of variants of the ReLU function that introduce a non-zero slope for negative inputs, allowing for a small gradient even when the input is negative. Formally, the Generalized ReLU can be expressed as:

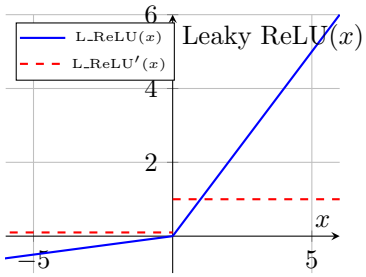
$$\text{GReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

where α is a hyperparameter that controls the slope for negative inputs. Different choices of α lead to different variants of the ReLU function, such as:

  - **Leaky ReLU**: a variant of ReLU that allows a small, non-zero gradient for negative values. It is defined as:

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

where $\alpha = 0.01$



  - **Parametric ReLU (PReLU)**: where α is a learnable parameter that is updated during training;
  - **Randomized ReLU (RReLU)**: where α is randomly sampled from a uniform distribution during training, and fixed to a value during testing;
  - **Exponential Linear Unit (ELU)**: where the function is defined as:

$$\text{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

This has all the benefits of ReLU while avoiding the dying ReLU problem, but requires more computational resources due to the exponential function.

All this proposed solutions have the common property of being monotonic for positive values. Recently, non-monotonic activation functions have been proposed, such as:

- **Swish**: defined as $f(x) = x \cdot \sigma(x)$, where $\sigma(x)$ is the sigmoid function. The resulting function is non-monotonic and smooth everywhere, which has shown to perform better than ReLU in some cases, but it is more computationally expensive due to the sigmoid function.

- **GelU**: defined as $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is an approximation of the cumulative distribution function of the standard normal distribution. This function integrates the properties of ReLU and regularizations techniques, such as dropout and zoneout.

2.7 Architectures

When designing a FFNN, we need to make several choices regarding the architecture of the network, such as:

- **Number of Layers**: how deep should the network be?
- **Number of Neurons per Layer**: how many neurons should each layer have?
- **Expressiveness of the Network**: how complex should the network be to capture the underlying structure of the data?

We know that a 2-layer Neural network are universal function approximators, which means that they can approximate any continuous function to an arbitrary degree of accuracy, given enough neurons in the hidden layer. This implies that we could use a shallow network with a single hidden layer, by increasing the number of neurons in that layer (wide network).

However, in practice, it's not guaranteed that our training algorithm will find the optimal parameters that allow the network to approximate the target function, as we have to take into consideration the problems in optimization and generalization. It has been shown that deeper networks can generally capture more complex functions and achieve better generalization performance than shallower networks.

However, increasing the depth of the network is not always the best solution. Other approaches instead focused on architecutral design choices rather than depth. In particular, this idea was first explored in the context of Convolutional Neural Networks (CNNs), where the use of convolutional layers allows to get better performance with fewer parameters compared to FFNNs with fully connected layers.

2.8 Optimization

To train a FFNN, we need to optimize the parameters of the network to minimize the loss function. This is usually done using gradient-based optimization algorithms, such as gradient descent and its variants.

These algorithms are based on the idea of iteratively updating the parameters of the network in the direction of the steepest descent of the loss function, which

is given by the negative of the gradient of the loss function with respect to the parameters. To better understand this, let's first introduce the concept of gradients.

2.8.1 Gradients

Gradients are a vector of partial derivatives that indicate the **direction** and **magnitude** of the **steepest ascent** of a function. Given the loss function $\mathcal{L}(\boldsymbol{W})$, the gradient with respect to the parameters w_i is given by:

$$\nabla_{w_i}\mathcal{L} = \left[\frac{\partial\mathcal{L}}{\partial w_1}, \frac{\partial\mathcal{L}}{\partial w_2}, \ldots, \frac{\partial\mathcal{L}}{\partial w_N}\right]$$

Each component of the gradient vector $\frac{\partial\mathcal{L}}{\partial w_i}$ measures the rate of change of the loss function with respect to each parameter.

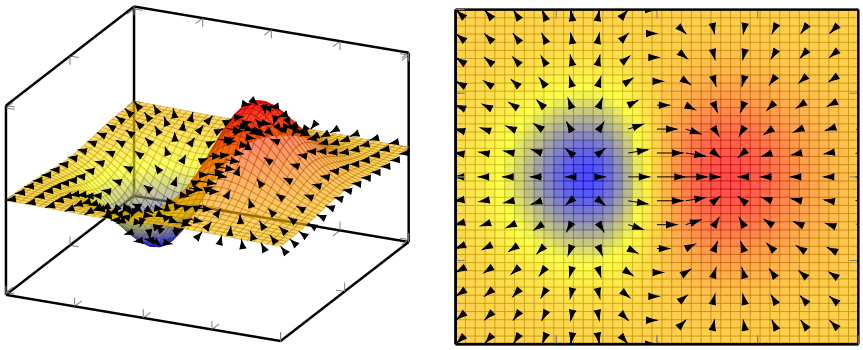


Figure 10: Visualization of derivatives and gradients.

If we move on the opposite side of the gradients, we will be moving in the direction of the **steepest descent** of the loss function. This way, these algorithms can iteratively explore the parameter space to find the optimal parameters that minimize the loss function.

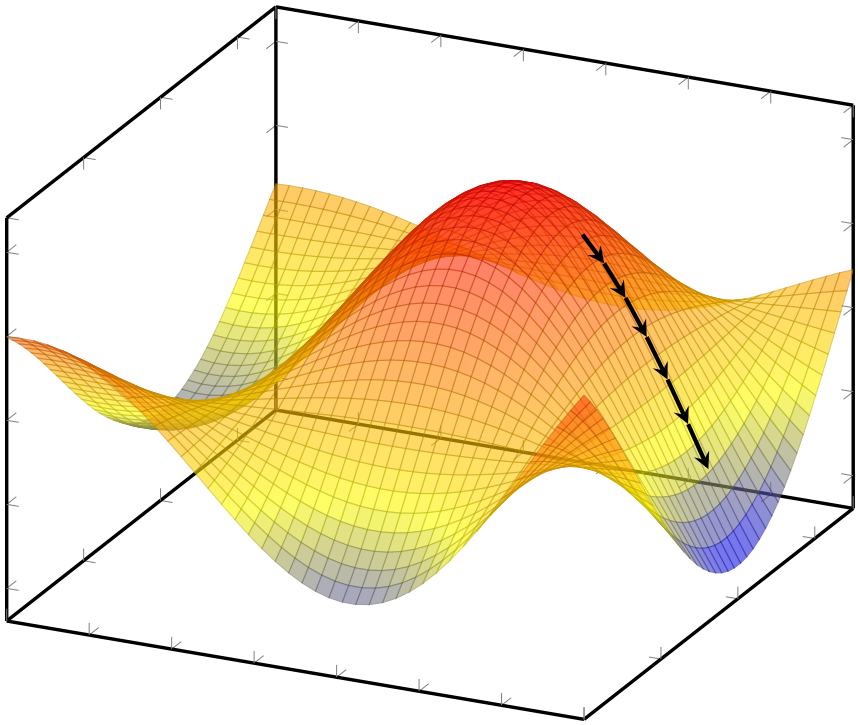


Figure 11: Visualization of Gradient Descent.

2.8.2 Problems with gradients

By now we know that in neural networks, the loss function is usually non-convex, which means that it can have multiple local minima and saddle points.

Local minima Local minima are points in the parameter space where the loss function has a lower value than in the surrounding area, but it is not the global minimum. This can lead to suboptimal performance of the model.

Saddle points Similarly, saddle points are points in the parameter space where the loss function has a gradient of zero, but does not correspond to a local minimum.

Both local minima and saddle points are issues that can arise during the optimization process. Altough they can lead to suboptimal performance, there are still other issues regarding optimization, that are much more important.

2.8.3 Gradient Descent

Batch Gradient Descent

Batch Gradient Descent is an optimization algorithm used to minimize the loss function by iteratively updating the parameters of the model in the direction of the negative gradient of the loss function with respect to the parameters.

In the general case, we can express the update rule for the parameters $\boldsymbol{\theta}$ as follows:

Algorithm 1 Batch Gradient Descent

Require: Learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
while stopping criteria not met **do**
  Compute **gradient** estimate over the entire dataset:
  $\nabla \leftarrow \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

where η is the learning rate, which controls the step size of the updates and some parameter initialization is required to start the optimization process.

This allows for more stable gradient updates. However, computing the gradient over the entire dataset can be computationally expensive, especially for large datasets.

Remarks on Learning Rate

For better result, the learning rate η should be adjusted during training, starting with a higher value and gradually decreasing it as the optimization process progresses. This can be defined trough the Learning Rate Schedule:

$$\eta_k = (1 - \alpha)\eta_0 + \alpha\eta_\tau \quad \alpha = \frac{k}{\tau}$$

The starting value of the learning must be choosen by trying different values and analyzing the learning curves on the validation set.

- If the Learning Rate is too large, the curves will show big oscillations.
- If it is too low, learning proceeds too slowly.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is a variant of gradient descent that updates the parameters using only a single training example at a time, allowing for faster updates and convergence. The update rule for SGD can be expressed as follows:

Algorithm 2 Stochastic Gradient Descent

Require: Learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

This allows for faster updates and convergence, but causes a non-smooth optimization process. This is not always a bad thing, as it can help to escape local minima and saddle points, but it can also lead to more noisy updates and slower convergence.

Mini Batch

To mitigate the issues of both Batch Gradient Descent and Stochastic Gradient Descent, we can use Mini Batch Gradient Descent. Instead of using the entire dataset or a single training example, we can use a small batch of training examples to compute the gradient and update the parameters. The size of the mini batch is a hyperparameter that can be tuned based on the specific problem, dataset size, and hardware constraints.

This has many advantages. First, its computation isn't dependent on the size of the dataset anymore, allowing computations on extremly large datasets. Secondly, it allows for parallelization of the computations, as we can compute the gradients for each mini batch in parallel, which can significantly speed up the training process. Finally, it provides a good balance between the stability of Batch Gradient Descent and the speed of Stochastic Gradient Descent.

However, along flat direction of the loss function, the gradient steps can be very small, leading to slow convergence. In steep ravines, the gradient steps can be very large, leading to oscillations and divergence. To mitigate these issues, the concept of momentum was introduced.

2.8.4 SGD with Momentum

To solve the issues of slow convergence along flat directions and oscillations in steep ravines, we can introduce the concept of momentum into the SGD algorithm.

The idea is to introduce a **velocity** term v that accumulates the gradients over time, allowing for faster convergence and smoother updates. This can be represented by exponentially decaing moving average of the negative gradients, which can be expressed as follows:

$$v \leftarrow \alpha v - \eta \nabla_{\boldsymbol{\theta}} (\mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}))$$

Where α is the momentum hyperparameter that controls the contribution of the previous velocity to the current update.

The update rule for the parameters can then be expressed as follows:

Algorithm 3 Stochastic Gradient Descent with Momentum

Require: Learning rate η
Require: Momentum hyperparameter α
Require: Initial velocity v
Require: Initial parameters $\boldsymbol{\theta}$
while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $v \leftarrow \alpha v - \eta \nabla$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + v$
end while
Output: Learned parameters $\boldsymbol{\theta}$

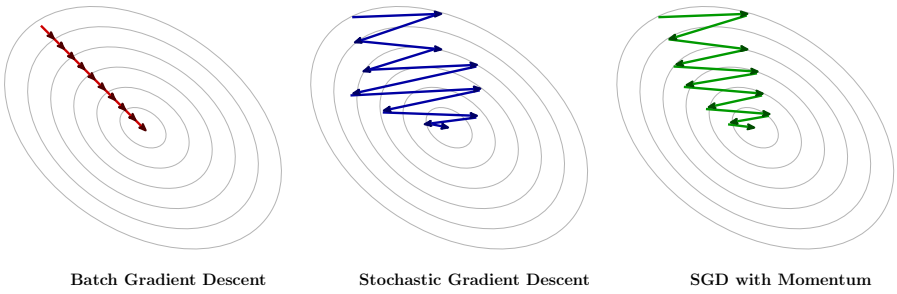


Figure 12: Different optimization algorithms.

SGD with Nesterov Momentum

Nesterov Momentum is a variant of the momentum method that provides a look-ahead mechanism to improve convergence. It works by first taking a step in the direction of the previous velocity (momentum) and then computing the gradient at the new position, which allows for a more informed update of the parameters.

This helps because, while the gradient always point to the right direction, the momentum can sometimes lead to overshooting or oscillations. The look-ahead mechanism of Nesterov Momentum allows to mitigate these issues by providing a correction to the momentum update on the same step.

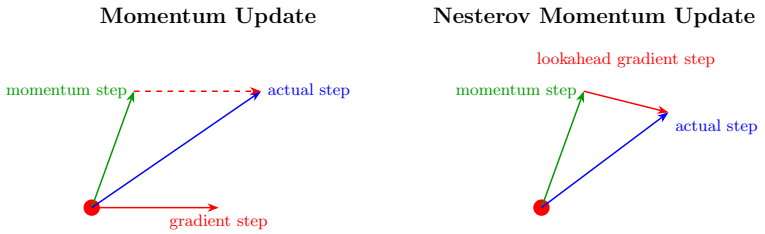


Figure 13: Different optimization algorithms.

The update rule for Nesterov Momentum can be expressed as follows:

Algorithm 4 Stochastic Gradient Descent with Nesterov Momentum

Require: Learning rate η
Require: Momentum hyperparameter α
Require: Initial velocity v
Require: Initial parameters $\boldsymbol{\theta}$
while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta} + \alpha v), y^{(i)})$
  $v \leftarrow \alpha v - \eta \nabla$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + v$
end while
Output: Learned parameters $\boldsymbol{\theta}$

This look-ahead mechanism allows Nesterov Momentum to achieve faster convergence and slightly better performance compared to standard momentum.

2.8.5 Adaptive Learning Rates Methods

So far, we have seen optimization algorithms that use a fixed learning rate η for all parameters. This means that each of the parameters has the same importance during the optimization process, which is not always the case.

Adaptive learning rate methods are a family of optimization algorithms that adjust the learning rate for each parameter based on the historical gradients.

Adagrad

Adagrad is an optimization algorithm that adapts the learning rate for each parameter by scaling it inversely proportional to the square root of the accumulated squared gradients.

In other words, it assigns a separate learning rate to each parameter based on how frequently it has been updated in the past. Parameters with large or frequent gradients receive smaller effective learning rates, while parameters with infrequent or small gradients receive larger learning rates.

The update rule for Adagrad can be expressed as follows:

Algorithm 5 Adagrad

Require: Initial learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
Require: Initial accumulated squared gradients $r = 0$
while stopping criteria not met **do**
  $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
  $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
  $r \leftarrow r + \nabla^2$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{r}+\delta} \nabla$
end while
Output: Learned parameters $\boldsymbol{\theta}$

where δ is a small constant added to prevent division by zero.

Adagrad started was the first adaptive learning rate method, that started a real trend in Deep Learning, but it has some issues. In particular, the weight decay can be too aggressive, as the accumulated squared gradients can grow very large, leading to very small effective learning rates and slow convergence.

RMSProp

The problem of Adagrad can be mitigated in non-convex settings by replacing the accumulated squared gradients with an exponentially decaying average of the gradient.

In non-convex optimization, the Adagrad accumulator r grows indefinitely, eventually shrinking the learning rate to a point where the model stops learning before reaching a local minimum. RMSProp "forgets" some of the past, allowing it to adapt to the local curvature of the loss surface.

The update rule for RMSProp can be expressed as follows:

Algorithm 6 RMSProp

Require: Initial learning rate η
Require: Initial parameters $\boldsymbol{\theta}$
Require: Initial accumulated squared gradients $r = 0$
1: **while** stopping criteria not met **do**
2: $i \leftarrow$ randomly selected index from $\{1, 2, \ldots, N\}$
3: $\nabla \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
4: $r \leftarrow \beta r + (1 - \beta)\nabla^2$
5: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{r}+\delta}\nabla$
6: **end while**
7: **Output:** Learned parameters $\boldsymbol{\theta}$

where β is a decay factor.

It's important to note that there is also a variant of RMSProp that includes the Nesterov Momentum, which can further improve convergence and performance.

Adam

Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of both momentum and adaptive learning rates. It computes adaptive learning rates for each parameter by keeping an exponentially decaying average of both the gradients and the squared gradients.

2.8.6 Batch Normalization

Batch Normalization is a technique introduced to resolve the problem of internal covariate shift. As learning proceeds, the distribution of each layer's inputs change as the parameters of the previous layers change. This can slow down the training process, as the network needs to continuously adapt to the changing input distribution.

Batch Normalization allows to normalize the inputs of each layer to have zero mean and unit variance, which helps to stabilize the learning process and allows for faster convergence.

Standard Batch Normalization To perform batch normalization, we first need to compute the mean and standard deviation of the inputs. Suppose that H is the input of a mini-batch to a layer, we can compute the mean and standard deviation as follows:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} H_i$$

$$\sigma_B = \sqrt{\delta + \frac{1}{m}\sum_{i=1}^{m}(H_i - \mu_B)^2}$$

To standardize the inputs, we can then substitute the original inputs H with the normalized inputs $\hat{H}$, which can be expressed as follows:

$$\hat{H} = \frac{H - \mu_B}{\sigma_B}$$

Learnable Parameters While standard batch normalization helps to stabilize the learning process, standardizing the output of units can limit the expressiveness of the network.

To solve this problem, we can introduce learnable parameters γ (scale) and β (bias) that allow to scale and shift the normalized inputs. The update of the feature matrix H can then be expressed as follows:

$$H' = \gamma\hat{H} + \beta$$

This allows the network to learn a linear transformation of the normalized inputs, which can help to improve the expressiveness of the network while still benefiting from the stabilization provided by batch normalization.

Training and Testing During training, the backpropagation algorithm is used to update the parameters γ and β based on the gradients of the loss function with respect to these parameters.

During testing, the running avareges of μ and σ collected during training and used to normalize the inputs, instead of computing the mean and standard deviation from the mini-batch of test data. This is done to ensure that each example is threated independently during testing.

Properties Batch Normalization was firstly introduced to solve the problem of internal covariate shift, but many other properties of this technique have been discovered since then, such as:

- Improves gradient flow through the network, allowing for faster convergence and better performance;

- Allows for higher learning rates, which can further speed up the training process;
- Reduces the strong dependence on the initialization of the parameters;
- Acts as a regularizer, reducing the need for other regularization techniques.

Layer Normalization Other normalization schema have been proposed to solve the problem of internal covariate shift, such as Layer Normalization.

Instance Normalization

2.9 Backpropagation

The perceptron learning algorithm (see 2.1) is not suitable for training FFNNs, as it requires to compute the difference between the predicted output and the ground truth for each training example. This is not possible for FFNNs, as in fact for hidden layers we don't have access to the true output, and we need to compute the error based on the output of the network and the true target variable.

To solve this problem, we can use the **Stochastic Gradient Descent** (SGD) algorithm, in combination with the **backpropagation** algorithm, which allows to efficiently compute the gradients of the loss function with respect to the parameters of the network.

2.9.1 Idea behind Backpropagation

Before backpropagation, a few ideas were proposed to learn the weights inside a FFNN:

- **Randomly perturb the weights**: randomly perturb one weight at a time and check if the loss function decreases. This is inefficient and does not scale well with the number of parameters in the network.
- **Parallel Perturbation**: randomly perturb all the weights at the same time and check if the loss function decreases. This doesn't help at all, as we would need many perturbations to get a good estimate of the gradient;
- **Perturb the activations**: randomly perturb the activations of the hidden layers and check if the loss function decreases. This is more efficient than perturbing the weights, as we have fewer activations than weights, but it still does not scale well with the number of parameters in the network.

From the last idea, the backpropagation algorithm was born, which instead measure how the loss function changes with respect to the activations of the hidden layers.

2.9.2 The algorithm

The backpropagation algorithm, introduced in the section 2.1, consists of three main steps:

- **Forward Pass**: compute the output of the network for a given input $\boldsymbol{x}$ and the current parameters $\boldsymbol{\theta}$;
- **Error Computation**: compute the error at the output layer by comparing the predicted output with the true target variable y;
- **Backward Pass**: propagate the error signal back through the network, allowing to update the parameters of the network using the computed gradients.

As said before, we cannot know from the training data what the expected output of the hidden units should be, but we can compute how the error changes with respect to the change in the hidden units.

2.9.3 Single Neuron Case

Let's first consider a neural network with just one neuron in each layer. After performing the forward pass, we can express the output of the network as follows:

$$a^{(L)} = \sigma(z^{(L)}) = \sigma(w^{(L)}a^{(L-1)} + b^{(L)})$$

where $a^{(L)}$ is the output of the last layer, σ is a non-linear activation function, $w^{(L)}$ and $b^{(L)}$ are respectively the weight and bias of the last layer, and $a^{(L-1)}$ is the output of the previous layer.

Our goal is to understand how a small change in the parameters $w^{(l)}$ will affect the loss function $\mathcal{L}$. When computing this, we need to consider the chain of dependencies between the parameters and the loss function:

- How a change in $w^{(l)}$ affects the output of the layer $z^{(l)}$;
- How a change in $z^{(l)}$ affects the output of the layer $a^{(l)}$;
- How a change in $a^{(l)}$ affects the loss function $\mathcal{L}$.

This is where the chain rule of calculus comes into play, allowing us to compute the gradient of the loss function with respect to the parameters by multiplying the gradients along the chain of dependencies.

Formally, we can express the gradient of the loss function with respect to the parameters $w^{(l)}$ as follows:

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \frac{\partial z^{(l)}}{\partial w^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a^{(l)}}$$

where respectively:

- $\frac{\partial z^{(l)}}{\partial w^{(l)}}$ measures how a small change in the weight $w^{(l)}$ affects the output of the layer $z^{(l)}$;
- $\frac{\partial a^{(l)}}{\partial z^{(l)}}$ measures how a small change in the output of the layer $z^{(l)}$ affects the output of the activation function $a^{(l)}$;
- $\frac{\partial \mathcal{L}}{\partial a^{(l)}}$ measures how a small change in the output of the activation function $a^{(l)}$ affects the loss function $\mathcal{L}$.

The process can be extended to deeper layers. In those cases, we need to consider the chain of dependencies from the parameter we want to update to the output of the network, computing the gradients along the way using the chain rule of calculus.

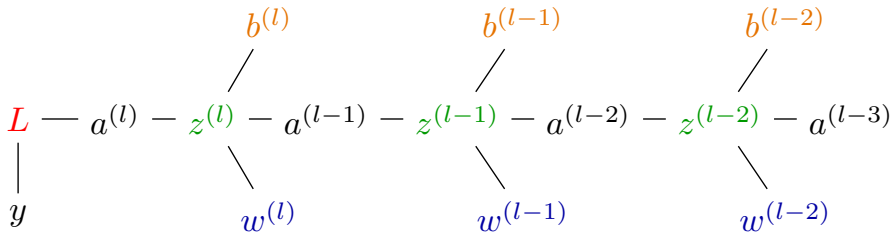


Figure 14: Dependency chain for a parameter in a deep network.

Taking in consideration the architecture shown in the figure above, we can express the derivative of the loss function with respect to a parameter $w^{(l-1)}$ in a deeper layer as follows:

$$\frac{\partial \mathcal{L}}{\partial w^{(l-1)}} = \frac{\partial z^{(l-1)}}{\partial w^{(l-1)}} \cdot \frac{\partial a^{(l-1)}}{\partial z^{(l-1)}} \cdot \underbrace{\frac{\partial z^{(l)}}{\partial a^{(l-1)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a^{(l)}}}_{\frac{\partial \mathcal{L}}{\partial a^{(l-1)}}}$$

2.9.4 General Case

The general case of backpropagation doesn't differ much from the single neuron case. We simply have to consider all the possible way in which a change in a parameter can affect the loss function, and compute the gradients accordingly using the chain rule of calculus.

To take into account the fact that each layer can have multiple neurons, we also need to update the notation. Since each connection in the network has its own weight, we represent with $w_{ij}^{(l)}$ the weight connecting the j-th neuron in layer $l-1$ to the i-th neuron in layer l.

The weighted sum for the i-th neuron in layer l can be expressed as follows:

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

where $a_j^{(l-1)}$ is the output of the j-th neuron in the previous layer, and $b_i^{(l)}$ is the bias term for the i-th neuron in layer l.

When we consider the output layer neurons, the chain of dependencies remains the same as in the single neuron case, as each output neuron is directly connected to the loss function.

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(L)}} = \frac{\partial z_i^{(L)}}{\partial w_{ij}^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} \cdot \frac{\partial \mathcal{L}}{\partial a_i^{(L)}}$$

For the hidden layer neurons, we instead need to consider all the possible paths from the parameter we want to update to the output layer, and compute the gradients accordingly.

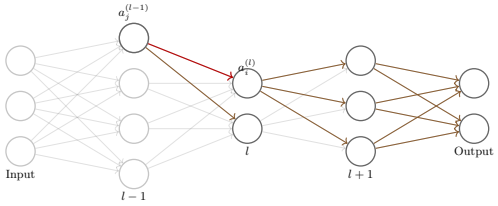


Figure 15: Dependency chain for a parameter feeding into h_2.

$$\frac{\partial \mathcal{L}}{\partial a_j^{(l-1)}} = \sum_i \frac{\partial z_i^{(l)}}{\partial a_j^{(l-1)}} \cdot \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \cdot \frac{\partial \mathcal{L}}{\partial a_i^{(l)}}$$

2.10 Regularization

Regularization is a set of techniques used to prevent overfitting in machine learning models, including FFNNs.

2.10.1 Model Capacity

Model capacity refers to the ability of a model to fit a wide variety of functions. We have that:

- **Low capacity models** may underfit the data, as they are not able to capture the underlying structure of the data;
- **High capacity models** may overfit the data, by memorizing properties of the training data that are not generalizable to unseen data.

We can control the capacity of a model by adjusting the hypotheses space, which is the set of all possible functions that the model can represent.

To deal with overfitting, we need to find a good balance between the capacity of the model and its ability to generalize to unseen data.

2.10.2 Simple approach

In addition to controlling the model capacity, many techniques have been proposed to control overfitting, some of which where already known in the context of machine learning, such as:

- **Collecting more data**: acquiring more training data can help to reduce overfitting, as it allows to better represent the underlying distribution of the data. However, this is not always a valiable option, as it might be expensive to acquire more data;
- **Ensemble methods**: combining the predictions of multiple models can help to reduce overfitting, as it allows to average out the errors of individual models;

2.10.3 Regularization Methods

For deeper architectures, controlling the complexity of the model is not as simple as adjusting the number of parameters. Many techniques have been proposed to regularize such complex models, without reducing their capacity. Commonly, many regularization methods can be combined to achieve better performance, as they often have complementary effects on the model.

In this section, we will discuss some of the most common regularization methods used in the context of FFNNs.

Parameter Norm Penalties One of the most common regularization techniques is to add a penalty term to the loss function that encourages the model to have smaller weights. The general formulation of this regularization technique can be expressed as follows:

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda\Omega(\boldsymbol{\theta})$$

where $\mathcal{L}(\boldsymbol{\theta})$ is the original loss function, $\Omega(\boldsymbol{\theta})$ is the regularization term, and λ is a hyperparameter that controls the strength of the regularization.

The most common choices for the regularization term $\Omega(\boldsymbol{\theta})$ are:

- **L2 Regularization**: also known as Ridge Regression, it adds a penalty term that encourages the model to have smaller weights by adding the squared norm of the weights to the loss function. The regularization term can be expressed as follows:

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\sum_i ||\theta_i^2||$$

- **L1 Regularization**: also known as Lasso Regression, it adds a penalty term proportional to the absolute value of the weights. Differently from L2 regularization, L1 regularization favors the sparsity of the weights, setting the weights of less important features to zero. The regularization term can be expressed as follows:

$$\Omega(\boldsymbol{\theta}) = \sum_i ||\theta_i||$$

Data Augmentation Data augmentation is a technique that involves creating new synthetic training examples by applying transformations to the original training data. This is based on the idea that generated images help the model generalize better to diverse data by exposing it to a wider variety of examples during training.

It is important to note that this method is primarily applicable to data types—such as images—where transformations can be applied without changing the identity of the object (invariance). For example, in image classification, we can apply transformations like rotation, scaling, and flipping, which do not change the image's class.

However, these transformations must be chosen carefully based on the task. For example, in the MNIST digit classification task, we cannot apply a 180 degree rotation to images of the classes "6" and "9," as it would change the class of the image.

Furthermore, data augmentation does not always guarantee better performance. While it introduces new samples, these examples may not add sufficient functional diversity to the training set since they are merely variations of existing data rather than entirely new information.

Noise Injection Noise injection is a regularization technique that involves adding random small perturbations to different levels of the network during training.

- **Input Noise**: Adding Gaussian noise to each input example during training can achieve effects similar to L_2 regularization. Each example can be represented as:
$$x_i' = x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$
where $\mathcal{N}(0, \sigma^2)$ is Gaussian noise with mean 0 and variance σ^2.

  For a linear layer, the noise variance is propagated through the weights. If $y = \sum w_i x_i$, the added variance at the node level is proportional to $\sum w_i^2 \sigma^2$. Consequently, to minimize the expected loss, the network is encouraged to maintain smaller weights, mimicking the effect of L_2 regularization. High levels of noise typically generate a smoother mapping function, improving generalization performance.

  This concept is also foundational to *Denoising Autoencoders*, where the network is trained to recover the original input from a corrupted version, forcing it to learn more robust latent representations.

- **Weight Noise**: This technique adds noise to the weights during the forward pass, forcing the model to remain performant even when parameters are slightly perturbed. This encourages the training process to converge to a *flat minimum* of the loss function, where the model's performance is less sensitive to small changes in weight values.

- **Output Noise**: This discourages the model from making overconfident predictions by smoothing out the label distribution (often referred to as label smoothing). This prevents the network from outputting extremely high logits for a single class, which can lead to better calibration and reduced overfitting.

Early Stopping A simple yet effective regularization technique is to stop the training process before the model starts to overfit the training data. This can be done by monitoring the performance of the model on a validation set during training, and stopping the training process when the performance on the validation set starts to degrade.

This procedure can be summarized as follows:

1. Start the training process with small weights;
2. Every time the error on the validation set decreases, save a copy of the model parameters;
3. When the training process is stopped, restore the model parameters to the values that achieved the best performance on the validation set.

Multi-Task Learning Multi-task learning is a regularization technique that involves training a model on multiple related tasks simultaneously.

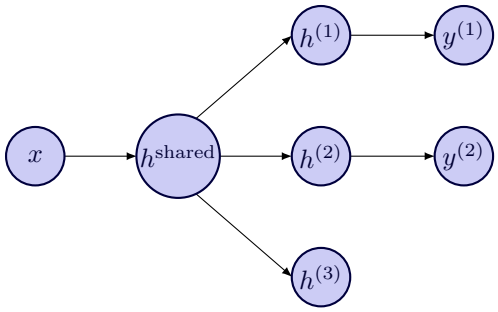


Figure 16: Example of a multi-task learning architecture. The model is trained on multiple related tasks simultaneously, sharing some intermediate representations.

Usually, we have several tasks that share the same set of input data $\boldsymbol{x}$, as well as some intermediate representations $h^{(shared)}$, but may have different output layers $f^{(1)}$, $f^{(2)}$, ..., $f^{(n)}$ for each task.

By sharing the intermediate representations, the model has 2 advantages:

- It can learn more robust and generalizable features, as it is forced to learn representations that are useful for multiple tasks;
- It can leverage the additional data from the other tasks, which can help to reduce overfitting and improve generalization performance.

Parameter Sharing Parameter tying is a regularization technique that involves sharing parameters across different parts of the model.

In parameter tying, we can enforce certain parameters to be closer to each other, or even to be exactly the same. This can be done by adding a penalty term to the loss function that encourages the parameters to be similar, or by explicitly tying the parameters together during the optimization process.

In parameter sharing, we can enforce certain parameters to be exactly equal to each other.

This technique can be particularly useful in cases where we have some prior knowledge about the structure of the data. In fact, it is at the base of *Convolutional Neural Networks* (CNNs), where the same set of weights is shared across different spatial locations in the input image, allowing the model to learn translation-invariant features.

Dropout We have already seen that the ensemble of multiple models can help to reduce overfitting by averaging out the errors of individual models. In deep learning, this approach would be computationally and time-consuming, as it would require training multiple models independently.

Dropout is a regularization technique born from the idea of ensemble learning, which allows to approximate the effect of training multiple models by randomly dropping out a subset of the neurons during training.

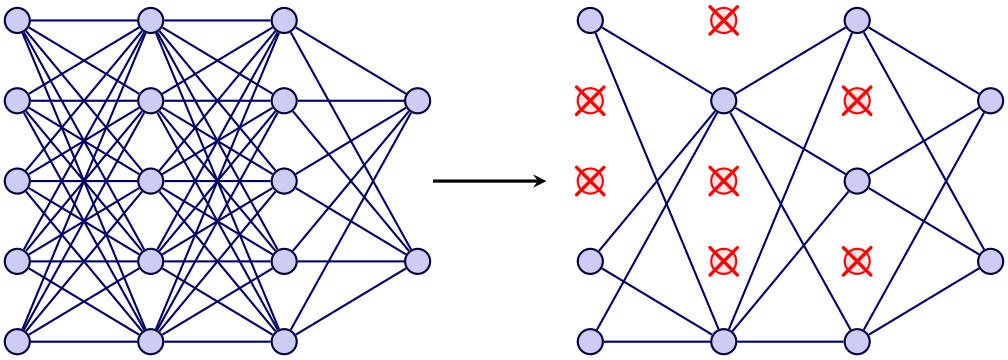


Figure 17: Example of a dropout regularization technique.

During the training process, each neuron is randomly dropped out (set to zero) with a certain probability p, which is a hyperparameter that controls the strength of the regularization. This forces the model to learn redundant representations, which allows the model to be more robust and less sensitive to the specific weights of individual neurons, thus improving generalization performance.

Dropout is similar to a *bagging* approach, where we train multiple models on different subsets of the data and average their predictions. We can see each trainging iteration as training a different model. At test time, we use the full network, but we scale the weights of the neurons by the dropout probability p to account for the fact that during training, each neuron was only active with probability p.

Implicit Regularization Some techinques, such as Stochastic Gradient Descent (SGD), have been shown to have an implicit regularization effect on the model. In particular, its stochastic nature can favor flatter minima of the loss function, which are associated with better generalization performance.

Also, using loss functions that are less sensitive to outliers, such as the Huber loss, can have an implicit regularization effect by reducing the influence of outliers on the training process.

3 Convolutional Neural Network

Some real world application of AI, such as image recognition, naturally work with an input space that is locally structured. Convolutional Neural Networks (CNNs) are a type of neural network architecture that is particularly well-suited for processing data with a 2D structure, such as images.

Similarly to FFNNs, the idea for CNNs has some biological inspiration, as they are loosely based on the structure of the visual cortex in animals. Neurons are organized in a hierarchical manner, with each layer processing increasingly complex features of the input image, starting from simple edges and textures in the early layers to more complex shapes and objects in the deeper layers.

If we were to use a standard FFNN for processing images, we would need to flatten the 2D image into a 1D vector, which would result in a loss of spatial information and a large number of parameters to learn, making the model prone to overfitting. CNNs are designed on 2 principles that allow them to effectively process images while keeping the number of parameters manageable:

- **Local Connectivity**: each neuron in a CNN is connected only to a small region of the layer before it;
- **Parameter Sharing**: the same set of weights is shared across different spatial locations in the input image. This allows the model to learn translation-invariant filters, while also significantly reducing the number of parameters to learn.

3.1 Convolution

Convolutional Neural Networks basically replace general matrix multiplications with convolution operations in at least one layer of the network.

1-D Convolution

Even though CNNs are typically used for processing 2D images, it's important to remember that the convolution operation can be applied to any type of data, including 1D sequences (e.g., time series, text)

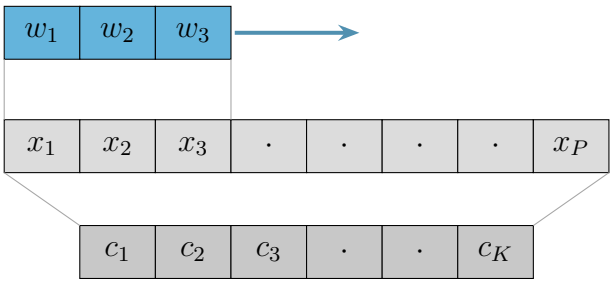


Figure 18: Example of a 1D convolution operation.

Given the example shown in the figure 18, we can express the convolution operation as follows:

$$c_1 = w_1x_1 + w_2x_2 + w_3x_3$$

$$c_2 = w_1x_2 + w_2x_3 + w_3x_4$$

And so on, where w_i are the weights of the filter and x_i are the input values.

2-D Convolution

In the case of 2D convolution, the input is a 2D image, and the convolution operation is performed by sliding a small filter (also called kernel) across the image and computing the dot product between the filter and the local region of the image. Formally, the output of a convolution operation for a given input image I and filter K is defined as:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j)$$

where $*$ denotes the convolution operation.

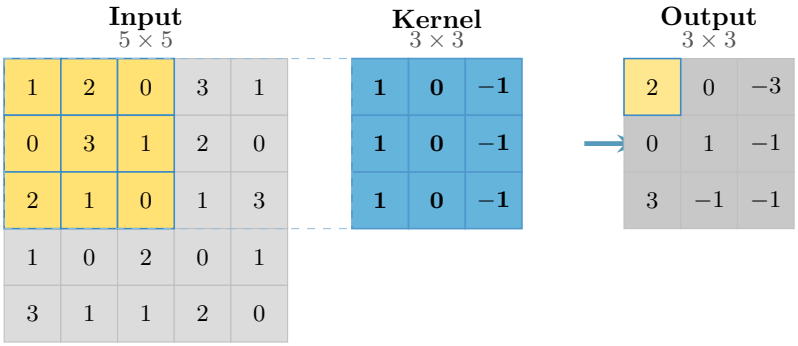


Figure 19: Example of a 2D convolution operation.

In the figure 19, we can see an example of a 2D convolution operation, where the input image is a 5×5 grid of pixel values, and the filter is a 3×3 kernel with specific values. The output of the convolution operation is a new 3×3 grid of values, where each value is computed as the dot product between the kernel and the corresponding local region of the input image.

3.2 Inside a CNN

A CNN is typically composed of several stacked layers, each performing a specific operation on the input data. Usually, low-level layers of the network learn to extract simple local features (e.g., edges, textures), while deeper layers learn to combine these features into more complex and abstract representations (e.g., shapes, objects).

Different types of layers can be used in a CNN, each one serving a specific purpose. In the following sections, we will briefly describe the most common types of layers used in CNNs.

3.2.1 Convolution Layer

The convolution layer is the core building block of a CNN, where the convolution operation is performed to extract features from the input data. Each layer consists of a set of filters (or kernels), covering a specific region of the input image, called the receptive field. These filters are applied across the entire input image, through the process of convolution, to produce a feature map that highlights the presence of specific features in the input image.

Differently from what was happening with feature extraction, where filters were designed manually, in CNNs the network learns the best filters for the task at hand through backpropagation and gradient descent, allowing it to automatically discover relevant features from the data.

It's important to note that the convolution layer has some hyperparameters that can be tuned to control the behavior of the convolution operation, such as:

- **Kernel Size**: the size of the filter (e.g., 3×3, 5×5) that determines the size of the receptive field and the amount of local information captured by the convolution operation.
- **Stride**: the step size with which the filter is moved across the input image. For example, a stride of 1 means that the filter is applied at every possible position, while a stride of 2 means that the filter is applied at every other position, effectively downsampling the output feature map.
- **Padding**: the output feature map produced by the convolution operation is smaller than the input image, due to the size of the kernel. To preserve

the spatial dimensions of the input image, we can add padding (e.g., zeros) around the borders of the input image before applying the convolution operation.

3.2.2 Non-Linearity Layer

After the convolution layer, a non-linearity layer is typically applied to introduce non-linear transformations to the feature maps. This is usually done using activation functions such as ReLU (Rectified Linear Unit), similar to what we have seen in FFNNs. The non-linearity layer allows the network to learn more complex and non-linear relationships between the input data and the output.

3.2.3 Pooling Layer

The pooling layer is used to downsample the feature maps produced by the convolution layer, reducing their spatial dimensions while retaining the most important information.

This is typically done using operations such as max pooling or average pooling, which respectively take the maximum or average value from a local region of the feature map, to produce a smaller output feature map.

This also helps to create the translation invariance property of CNNs, as the exact position of the features in the input image becomes less important after pooling.

3.3 Different Architectures

In this section, we will briefly describe some of the most popular CNN architectures that have been proposed in the literature, each one with its own unique design and characteristics.

3.3.1 LeNet (1998)

The LeNet architecture, proposed in 1998, is one of the earliest CNN architectures and was designed for handwritten digit recognition.

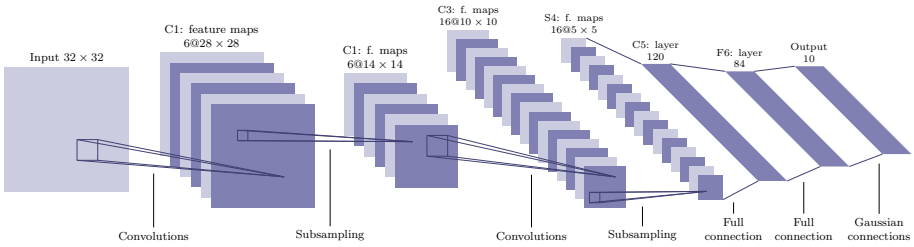


Figure 20: LeNet architecture.

It follows a simple architecture pattern, using 2 convolutional layers (C1 and C3), each followed by a pooling layer (S2 and S4). Finally, 2 fully connected layers (C5 and F6) are used to produce the final output, which is a probability distribution over the 10 possible digit classes (0-9).

3.3.2 AlexNet (2012)

Introduced for the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC), AlexNet significantly evolved the foundational concepts of LeNet. While it retained a similar structural logic, it scaled the complexity to 5 convolutional layers and 3 fully connected layers.

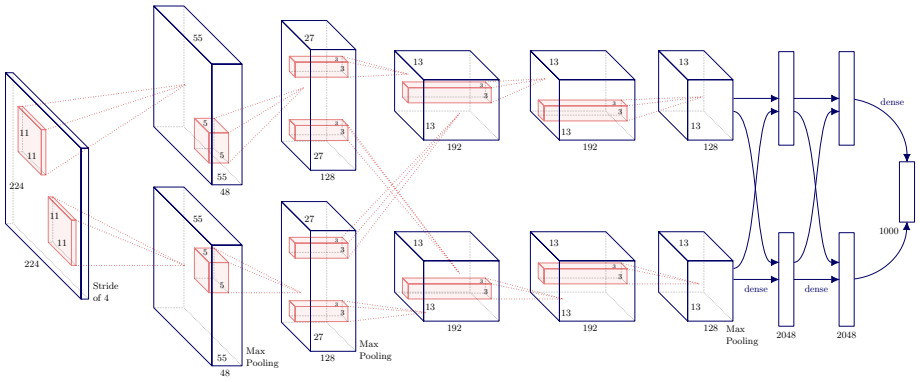


Figure 21: AlexNet architecture.

This architecture marked a turning point for deep learning. By dramatically outperforming existing methods on ImageNet, AlexNet demonstrated the power of deep convolutional networks and guided the research community towards deeper and more complex architectures.

Beyond its performance, AlexNet fundamentally changed how features were perceived. Previously, image processing relied on hand-crafted descriptors like SIFT or GIST paired with classifiers such as SVMs. AlexNet proved that an end-to-end trained CNN could automatically learn hierarchical features that were far more robust. Researchers soon discovered that these learned representations were highly transferable, outperforming manual features in diverse applications—including object detection, segmentation, and natural language processing.

3.3.3 Zeiler and Fergus (2013)

After the success of AlexNet, a natural question was: how can we understand what the network has learned?

The Zeiler and Fergus architecture, proposed in 2013, introduced a method for visualizing and understanding the features learned by deep convolutional networks. This approach involved deconvolutional networks, that allowed to invert the operations of a CNN, visualizing the activations of the different layers and understanding which features were being captured by the network.

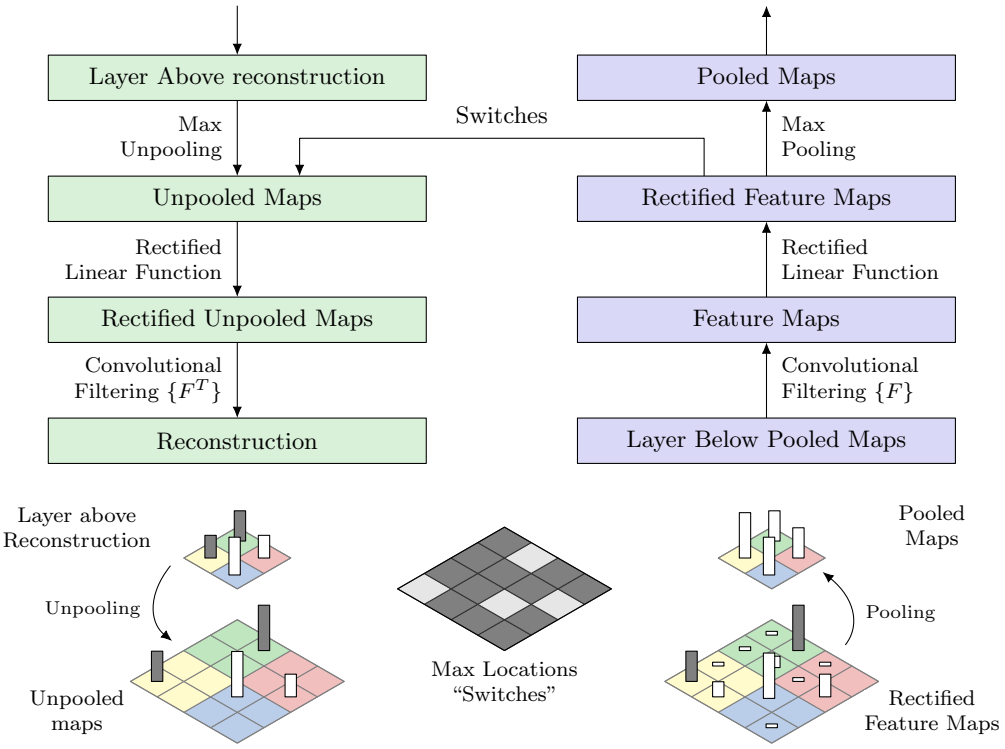


Figure 22: Zeiler and Fergus architecture for visualizing learned features.

How it works The deconvolutional network is designed to reverse the operations of a CNN, for each layer of the CNN. It is built using the same types of layers as a CNN, but in reverse order, and with the operations inverted. Obviously, the convolution operation and the activation functions are easily inverted, but the pooling operation is not invertible, as it removes information from the input feature map.

To address this issue, the authors introduced a method called "unpooling", which is used to approximate the inverse of the pooling operation. It basically uses the indices of the maximum values from the pooling operation to place the values back in their original positions, while filling the rest of the positions with zeros. This allows the deconvolutional network to reconstruct an approximation of the original input image, based on the activations of the different layers of the CNN.

Other approaches to explainability Other approaches to explainability in CNNs include:

- **Image Occlusion**: this method involves occluding different parts of the input image and observing how the output of the network changes, to understand which parts of the image are most important for the network's predictions;
- **Class Model Visualization**: more rigorous than the previous one, this method involves finding the input image that maximizes the activation of a specific class in the output layer, to understand which features are most important for that class;
- **Class Activation Mapping (CAM)**: uses each weight of the output layer to create a saliency map that highlights the regions of the input image that are most important for the network's predictions. The images are then averaged to create a heatmap that shows the most important regions of the input image for the network's predictions.

  This approch only works for fully convolutional networks, with no fully connected layers.
- **Grad-CAM**: An extension of CAM that can be applied to any CNN architecture. It uses the gradients of the output with respect to the feature maps of a specific convolutional layer to create a heatmap that highlights the important regions of the input image for the network's predictions.

3.3.4 VGG Net(2014)

VGG Net, follow the same architecture pattern of AlexNet, but with a much deeper architecture, using up to 19 layers. Many configurations of VGG Net

were proposed, with different numbers of convolutional layers and fully connected layers, but all of them were suffering from the same problem: the number of parameters was becoming too large (particularly in the fully connected layers), making the network prone to overfitting and increasing the computational cost.

Next architectures, proposed in the following years, used different architectural patterns to address this issue.

3.3.5 GoogLeNet (2014)

GoogLeNet, introduced in 2014, introduced the **Inception Module**, a network component that allows to capture features at multiple scales and levels of abstraction, by applying multiple filter sizes in parallel within the same layer.

The key ideas introduced by GoogLeNet were:

- **Parameter Efficiency:** By replacing computationally expensive fully connected layers with **Global Average Pooling**, GoogLeNet reduced the parameter count from approximately 60 million in AlexNet to just 6.7 million, despite being significantly deeper.
- **Depth:** The network consists of 22 layers, making it substantially deeper than its predecessors. This depth allows it to learn more complex and abstract features from the input data, contributing to its improved performance on the ImageNet dataset.
- **Dimensionality Reduction:** To keep the computational cost manageable, 1×1 convolutions are used to reduce the number of input channels before the expensive 3×3 and 5×5 convolutions.
- **Intermediate Supervision:** To mitigate the **vanishing gradient problem** caused by the increased depth, GoogLeNet introduced auxiliary classifiers at intermediate layers. These branches provide additional gradient signals back to the earlier layers during training.

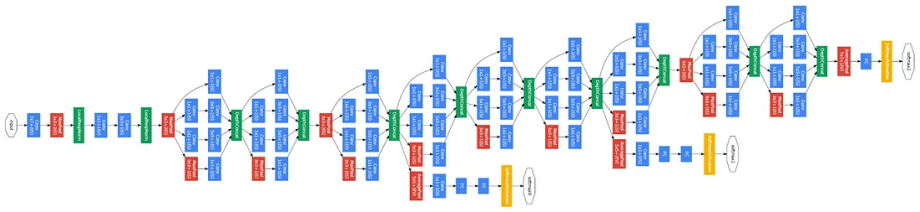


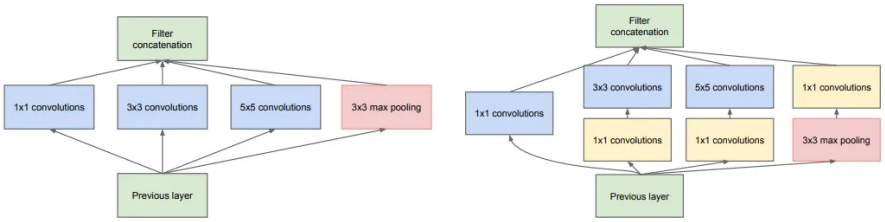
Figure 23: GoogLeNet architecture.

Inception Modules The Inception Module is a fundamental building block of the GoogLeNet architecture. It is designed to allow the network to learn multiple types of features at different scales, while keeping the number of parameters manageable. Instead of choosing a single filter size, the module applies 1×1, 3×3, and 5×5 convolutions, along with a 3×3 max-pooling operation, in parallel.

This approach offers two primary advantages:

- **Multiscale Feature Extraction:** It allows the network to automatically decide which filter size is most relevant for a given feature, effectively handling objects of varying sizes within the ImageNet dataset.
- **Feature Redundancy and Generalization:** Reusing the same input across parallel paths enforces a degree of redundancy in the learned feature maps. This redundancy encourages the model to learn more robust, scale-invariant representations, which significantly improves the generalization performance and provides a regularizing effect during training.

The outputs of all these operations are then concatenated together to produce the output feature map of the module.



(a) Inception module, naïve version (b) Inception module with dimension reductions

Figure 24: Inception Module.

3.3.6 ResNet (2015)

As architectures grew increasingly deep, researchers encountered the **degradation problem**: as network depth increases, accuracy begins to saturate and then degrades rapidly. Crucially, this degradation is not caused by overfitting, as it is characterized by higher **training error** than their shallower counterparts.

ResNet, introduced in 2015, addressed this issue by introducing the concept of **residual connections** (or skip connections), which allows the network to learn to skip certain layers if they are not useful for the task at hand, effectively allowing the network to learn an identity mapping (i.e., the input is passed through unchanged) if needed.

It was shown that this simple architectural provided various benefits, such as:

- **Implicit Ensemble Learning:** The skip connections allow the network to learn multiple paths for information flow, effectively creating an ensemble of shallower networks within the deeper architecture.
- **Smoother Loss Landscape:** The skip connections help to smooth the loss landscape, making it easier for the optimization algorithm to find a good solution and improving convergence during training.

Dimensions mismatch It's important to note that not always the skip connections could be used, due to dimension mismatch between the input and output of the layers. To resolve this issue, the concept of 1×1 convolution was reused, allowing to match the dimensions of the input and output feature maps, while also providing an additional level of feature transformation.

Different residual blocks In the later years, different types of residual blocks were proposed. Instead of identity skip connections, some architectures used more complex transformations in the skip connections, such as:

- **Pre-activation Residual Block**: the order of batch normalization and activation functions is changed, to precede the final convolution. This creates a cleaner identity mapping, as the skip connection is not affected by the non-linear transformations, and allows for better gradient flow during training;
- **Bottleneck Residual Block**: this block uses a 1×1 convolution to reduce the number of channels before applying the expensive 3×3 convolution, and then uses another 1×1 convolution to restore the original number of channels after the convolution operation.
- **Wide Residual Block**: focuses on increasing the width of the network (i.e., the number of channels) instead of the depth, by applying wider convolutional layers and using fewer layers overall. This approach allowed to investigate the trade-off between depth and width in CNNs, to find the optimal trade-off for different tasks and datasets.

Following the success of ResNet, many architectures were proposed that built upon the idea of residual connections. The next 2 architectures we will briefly describe are examples of this trend.

3.3.7 DenseNet (2018)

DenseNet are a type of CNN architecture that builds upon the idea of residual connections. Instead of using skip connections that add the input to the output of a layer, DenseNet introduces the **Dense Block**.

In a Dense Block, each layer is connected to all the other layers that compose the block. This means that for each layer, the feature maps of all the previous layers are concatenated together and used as input for the current layer.

This approach has a few advantages, such as:

- **Feature Reuse:** Since feature maps are preserved via concatenation, there is no need to relearn redundant features in subsequent layers.
- **Parameter Efficiency:** Despite the dense connectivity, DenseNets often require fewer parameters than ResNets for similar performance because they do not need large, wide layers to capture complex information.

3.3.8 SENet (2019)

SENet (Squeeze-and-Excitation Networks) is a type of CNN architecture that introduces the concept of **channel attention**. Channel attention allows the network to learn to focus on the most important channels of the feature maps, by applying a weighting mechanism to the channels based on their importance for the task at hand.

The SE block learns to emphasize informative channels and suppress less useful ones, effectively allowing the network to adaptively recalibrate channel-wise feature responses. This is achieved through a two-step process:

- **Squeeze**: Global average pooling is applied to the input feature, producing an average representation of each channel;
- **Excitation**: Using a sigmoid activation function, the network learns to generate weights for each channel.

The output of the SE block is obtained by multiplying the input feature maps by these learned weights, effectively emphasizing important channels and suppressing less useful ones.

3.3.9 ConvNext (2022)

In the later years, the trend of transformer-based architectures has dominated the field of computer vision, with models such as **Vision Transformer (ViT)** and **Swin Transformer** achieving state-of-the-art performance on various vision tasks.

ConvNext is a CNN architecture that was proposed in 2022. It showed that with some modifications to the standard CNN architecture, it is possible to achieve performance comparable to transformer-based architectures, while still maintaining the efficiency and simplicity of CNNs.

3.4 Regularization

To prevent overfitting, various regularization techniques were developed specifically for CNNs, such as:

- **Cutout**: this method involves randomly masking out a square portion of the input image during training, replacing it with the mean pixel value of the dataset. This approach forces the network to learn more robust features that are not reliant on specific parts of the image;
- **DropBlock**: an extension of dropout, where instead of randomly dropping individual neurons, contiguous regions of the feature maps are dropped during training, for all channels.
- **CutMix**: this method involves randomly cutting and pasting patches between training images. Class labels are also updated proportionally to the area of the patches. This encourages the model to learn features from different parts of the images, providing a regularization effect and improving generalization.

4 Expanding CNN Beyond Classification

In the previous chapter, we have seen how Convolutional Neural Networks (CNNs) can be used for image classification tasks. However, CNNs can be applied to a wide range of other tasks beyond classification, such as **object detection**, **semantic segmentation**, and **captioning**, among others. In this chapter, we will explore some of these applications and how CNNs can be adapted to solve them.

4.1 Object Detection

Object detection is the task of identifying and localizing objects within an image. It can be seen as a combination of classification (what object is present) and regression (where is the object located). The task input would be an image, and the output would be a set of bounding boxes around the detected objects, along with their corresponding class labels.

This task is more complex than image classification, as it requires the model to not only recognize the presence of objects but also to accurately localize them within the image. This makes also the training process more challenging, as several issues can arise, such as:

- **Data scarcity**: Object detection datasets are often smaller than classification datasets. Also, being the task more complex, it requires more data to train a model that can generalize well.
- **Poor localization**: the deep CNN proposed for classification may not be able to localize the objects accurately, because of their large receptive field and the pooling layers that reduce the spatial resolution of the feature maps.

To address these issues, several architectures have been proposed, such as **R-CNN**, **Fast R-CNN**, and **YOLO** (You Only Look Once), among others.

4.1.1 Before CNNs: DPM

Before AlexNet and the rise of CNNs, one of the most popular approaches for object detection was the Deformable Part Model (DPM). DPM is a method that models objects as a collection of parts, where each part can be described by a set of features. The model learns to detect these parts and their spatial relationships to identify the presence of an object in an image.

This approach was quite effective for its time, but reached a plateau in performance already in 2010, before the advent of deep learning.

4.1.2 Selective Search

One of the key challenges in object detection is to efficiently search for objects in an image. Selective Search is a method that generates a set of candidate regions (also called proposals) in an image that are likely to contain objects. Given the proposals, the process of object detection can be simplified to a classification task, where the model only needs to classify the proposals as containing an object or not, which is much easier than searching for objects in the entire image.

To detect objects at any scale, Selective Search uses a hierarchical grouping of superpixels, which are small regions of an image that have similar features (such as color, texture, or intensity). The algorithm starts by generating all the possible candidate objects. Similar regions are then merged together based on their similarity. Once the merging process is complete, the algorithm uses the resulting regions as proposals for object detection.

4.1.3 R-CNN

R-CNN (Region-based Convolutional Neural Networks) is one of the first architectures that successfully applied CNNs to object detection. The R-CNN architecture consists of three main steps:

- **Region Proposal**: The first step is to generate a set of candidate regions (proposals) in the image using Selective Search.
- **Feature Extraction**: The second step is to extract features from each proposal using a CNN.
- **Classification**: The final step is to classify each proposal as containing an object or not, and to assign a class label to the detected objects. This was typically done using a Support Vector Machine (SVM) classifier, which was trained on the features extracted from the proposals.

This approach was a significant improvement over previous methods, but had a few limitations, such as:

- **NOT an end-to-end architecture**: The R-CNN architecture is not an end-to-end architecture, as it requires separate training for the region proposal, feature extraction, and classification steps. Deep learning models are known to perform better when trained end-to-end, as they can learn to optimize the entire process jointly.
- **Image resizing**: Each proposal must be resized to a fixed size (e.g., 224x224) before being fed into the CNN for feature extraction. This can lead to a loss of information, especially for small objects, and can also introduce distortions in the aspect ratio of the proposals.

- **Computationally expensive**: The R-CNN architecture is computationally expensive, as it requires running the CNN for each proposal, which can be in the order of thousands per image (takes around 47 seconds per image on a GPU).

4.1.4 Fast R-CNN

Fast R-CNN improves upon the original R-CNN by addressing its heavy computational redundancy and disjointed training pipeline. Instead of processing thousands of individual region proposals through a CNN, it utilizes a **shared computation strategy**.

The main innovations of Fast R-CNN are:

- **Shared Convolutional Architecture**: Instead of running the CNN separately for each proposal, Fast R-CNN processes the entire image through a convolutional network to produce a global feature map.
- **RoI Pooling Layer**: To handle the variable sizes of the region proposals, Fast R-CNN introduces a Region of Interest (RoI) pooling layer. This layer takes the feature map generated from the entire image and extracts fixed-size feature vectors for each region proposal. The RoI pooling layer divides the feature map into a grid and applies max pooling to each grid cell, resulting in a fixed-size output regardless of the original proposal size.
- **Multi-Task Loss**: Fast R-CNN also introduces a multi-task loss function that combines classification and bounding box regression into a single optimization problem. This allows the model to learn both tasks simultaneously, improving the overall performance and efficiency of the training process.

This approach allowed reducing inference time significantly (to around 0.3 seconds per image on a GPU, from the 47 seconds of R-CNN), while also improving the accuracy of the model, while also allowing for end-to-end training of the entire architecture.

4.1.5 Faster R-CNN

While Fast R-CNN significantly reduced the computational cost of the detection head, it remained bottlenecked by external region proposal methods like **Selective Search**, which typically took ~ 2 seconds per image on a CPU.

Faster R-CNN addresses this by introducing a **Region Proposal Network (RPN)**, allowing for a truly unified, end-to-end object detection framework.

The architecture of Faster R-CNN consists of the following components:

- **Shared Convolutional Network**: Like its predecessor, it processes the entire image once to generate a high-level feature map.
- **Region Proposal Network (RPN)**: A small, fully convolutional network that slides over the feature map. At each sliding window location, it predicts objectness scores and bounding box offsets for k **anchor boxes** of varying scales and aspect ratios.
- **RoI Pooling Layer**: This layer crops and reshapes the regions proposed by the RPN from the shared feature map into fixed-size feature vectors.
- **Classification and Bounding Box Regression**: The final stage consists of fully connected layers that predict the specific object class and perform fine-grained coordinates refinement.

Performance By replacing Selective Search with the RPN, Faster R-CNN achieves an inference time of approximately **0.2 seconds** per image on a GPU. Furthermore, because the RPN is trained specifically for the task, it typically yields higher-quality proposals than traditional computer vision algorithms, leading to improved mAP.

You Only Look Once (YOLO) YOLO (You Only Look Once) is a different approach to object detection. Instead of using a two-stage process like R-CNN and its variants, which was very slow and hard to train, YOLO treats object detection as a single regression problem, directly predicting bounding boxes and class probabilities from the input image in one pass through the network.

The process of YOLO can be summarized as follows:

- **Grid Division**: The input image is divided into an $S \times S$ grid. Each grid cell is responsible for predicting m bounding boxes;
- **Bounding Box Prediction**: For each bounding box, the model predicts the coordinates of the box and a class confidence score that indicates the likelihood of the box containing an object and the class of the object;
- **Buonding Box Selection**: During inference, the model selects only the bounding boxes which have a confidence score above a certain threshold.

This design allows YOLO to achieve real-time object detection, with inference times as low as 0.02 seconds per image on a GPU, while still maintaining competitive accuracy compared to two-stage detectors. Even YOLO has some limitations, such as:

- **Small Object Detection**: YOLO can struggle with detecting small objects, as the grid cell responsible for predicting the object may not have enough information to accurately localize it.

- **Different Aspect Ratios**: YOLO uses a fixed set of anchor boxes, which may not be optimal for all object shapes and sizes, leading to suboptimal performance for objects with unusual aspect ratios.
- **Loss Function**: The original YOLO loss is very complex and can be difficult to optimize, as it combines classification and localization losses, which can have different scales and gradients.

4.2 Instance Segmentation

A different task that can be tackled with CNNs is **instance segmentation**. Instance segmentation is the task of assigning a class label to each pixel in an image, while also distinguishing between different instances of the same class. For example, in an image containing multiple people, instance segmentation would not only identify the pixels belonging to the "person" class but also differentiate between each individual person.

This task is more complex than object detection, as it requires not only localizing objects but also accurately segmenting them at the pixel level.

Mask R-CNN

One of the most popular architectures for instance segmentation is **Mask R-CNN**, which extends the Faster R-CNN architecture by adding a branch for predicting segmentation masks on each Region of Interest (RoI).

To achieve this, Mask R-CNN introduces:

- **RoIAlign Layer**: Instead of using the RoI Pooling layer from Faster R-CNN, which can quantize the RoI and lead to misalignments between the RoI and the extracted features, Mask R-CNN uses a RoIAlign layer that preserves the spatial alignment of the features, allowing for more accurate mask predictions.
- **Mask Branch**: In addition to the classification and bounding box regression branches, Mask R-CNN includes a separate branch that predicts a binary mask for each RoI. This branch is typically implemented as a small fully convolutional network that takes the aligned features from the RoIAlign layer and outputs a mask of fixed size for each class.

5 Recurrent Neural Networks

Unlike the architectures previously discussed, which only process data represented as fixed-size vectors for both input and output, **Recurrent Neural Networks** (RNNs) are designed to handle sequential data. Standard feed-forward models face significant limitations with real-world data types—such as text, audio, and video—which are inherently sequential and vary in length.

Recurrent Neural Networks (RNNs) were the first architectures specifically designed to handle this kind of data, enabling solutions to a wide range of tasks, such as:

- **Document classification**: Given a document, classify it into one of several categories (e.g., spam vs. non-spam). We know that the document size cannot be fixed a priori, as it can vary significantly from one document to another.
- **Sentiment analysis**: similar to document classification, but the goal is to classify the sentiment of a text.
- **Image captioning**: Given an image, generate a caption that describes the content of the image. This is a more complex task, as it requires the model to generate a sequence of words that describe the image, which can have variable length.
- **Dynamical Systems**: dynamical systems are systems that evolve over time, based on the current state, some external input, and a predetermined set of rules that define how the system transitions from one state to another. RNNs can be used to learn a non-fixed transition function that captures the underlying dynamics of the system, allowing for accurate predictions of future states based on past observations.

5.1 RNN Architecture types

While classical Feed-Forward Neural Networks (FFNNs) are categorized as **one-to-one** (or single-to-single) architectures—taking one input and producing one output—RNNs are classified based on how they map input sequences to output sequences. The primary types include:

- **Many-to-One**: This type of RNN takes a sequence of inputs and produces a single output. This is the case of document classification, where the input is a sequence of words and the output is a single class label.

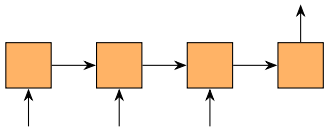


Figure 25: Many-to-One RNN architecture.

- **One-to-Many**: This type of RNN takes a single input and produces a sequence of outputs. This is the case of image captioning, where the input is an image and the output is a sequence of words that describe the image.

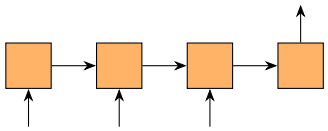


Figure 26: One-to-Many RNN architecture.

- **Many-to-Many**: This type of RNN takes a sequence of inputs and produces a sequence of outputs. We can have 2 subtypes of this architecture:
  - **Synchronized**: The input and output sequences have the same length, and an output is generated for every input (e.g., Frame-level video classification or Part-of-Speech tagging).
  - **Encoder-Decoder**: The model first processes the entire input sequence (encoding) before generating an output sequence of a potentially different length (decoding), as seen in Machine Translation.

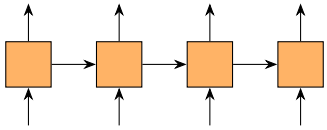


Figure 27: Many-to-Many Synchronized RNN architecture.

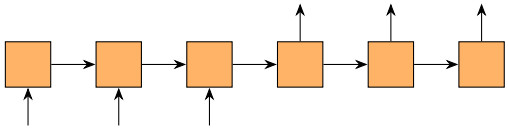


Figure 28: Many-to-Many Encoder-Decoder RNN architecture.

5.2 Vanilla RNNs

The simplest form of RNN is the **Vanilla RNN**, which consists of a single layer of recurrent units.

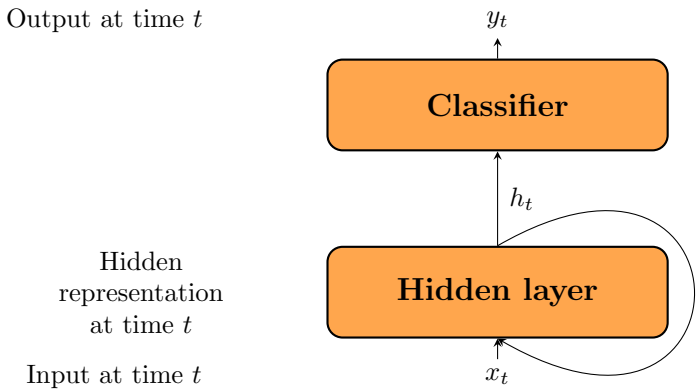


Figure 29: Vanilla RNN architecture.

At time step t, the model takes an input x_t, passes it through a hidden layer to produce a hidden representation h_t, which is then fed into a classifier to produce the output y_t.

At the next time step $t+1$, the model takes the next input x_{t+1} and also receives the hidden representation from the previous time step h_t as part of its input, allowing it to maintain a form of memory across time steps. This recurrent connection is what enables the model to capture temporal dependencies in the data, making it suitable for sequential tasks.

The hidden representation h_t can be computed as:

$$h_t = \sigma_W(x_t, h_{t-1}) \tag{5.1}$$

Where σ_W is a non-linear activation function (e.g., ReLU, Tanh) applied to a linear transformation of the current input x_t and the previous hidden state h_{t-1}, with W representing the learnable parameters of the model.

Usually, the choosen activation function is the hyperbolic tangent (tanh), which gives us:

$$h_t = \tanh W \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix} = \tanh(W_x x_t + W_h h_{t-1})$$

The RNN can also be unrolled in time, which means that we can represent the RNN as a feed-forward network where each layer corresponds to a time step in the sequence.

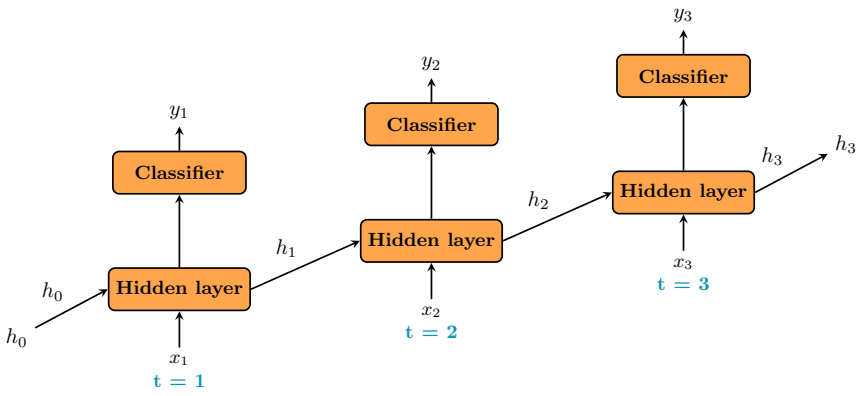


Figure 30: Unrolled RNN architecture.

5.2.1 Backpropagation Through Time

To train RNNs, we use a variant of backpropagation called **Backpropagation Through Time** (BPTT), which follows the same principles as standard backpropagation.

The unfolded RNN is treated as a deep feed-forward network, where each layer corresponds to a time step in the sequence. The weights are shared across all time steps, and the error is backpropagated through the entire sequence, allowing the model to learn from the temporal dependencies in the data.

Let's see a simple example of how BPTT works in a Vanilla RNN:

- **Forward Pass**: The input sequence (x_1, x_2, x_3) is fed into the RNN, producing hidden states (h_1, h_2, h_3) and outputs (y_1, y_2, y_3).
- **Loss Calculation**: The loss is computed based on the outputs and the true labels for each time step. The error is expressed as the sum of the losses at each time step:

$$L = \sum_{t=1}^{T} L_t(y_t, \hat{y}_t) \tag{5.2}$$

  In our example, with $T = 3$, we have:

$$L = L_1(y_1, \hat{y}_1) + L_2(y_2, \hat{y}_2) + L_3(y_3, \hat{y}_3) = L_1 + L_2 + L_3 \tag{5.3}$$

- **Backward Pass**: considering the weights associated with the hidden layer W_h, the gradient of the loss with respect to these weights is computed by summing the contributions from all time steps:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W_h} \tag{5.4}$$

Where each term $\frac{\partial L_t}{\partial W_h}$ is computed as:

$$\frac{\partial L_t}{\partial W_h} = \frac{\partial L_t}{\partial y_t} \cdot \frac{\partial y_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h} \tag{5.5}$$

For example, computing the gradient of the error with respect to h_1 (how the hidden state at time step 1 contributes to the loss) we have to consider:

- How h_1 contributes to h_2;
- How h_2 effects h_3;
- How h_3 contributes to the error at time step 3.

This gives us:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \tag{5.6}$$

5.2.2 BPTT issues

While BPTT allows RNNs to learn from sequential data, it suffers from the **vanishing gradient** problem. In fact, early time steps in the sequence receive gradients through a long chain of multiplications, which can lead to very small gradients that effectively prevent the model from learning long-term dependencies in the data.

This issue is particularly problematic for tasks that require understanding of long-range dependencies, such as language modeling or machine translation, where the model needs to capture relationships between words that are far apart in the input sequence.

5.2.3 Truncated BPTT

A common practical solution to the vanishing gradient problem and the high computational cost of standard BPTT is **Truncated Backpropagation Through Time** (TBPTT). The core idea is to process the sequence in smaller chunks rather than the entire history at once.

The algorithm can be characterized by two parameters:

- k_1: The number of time steps between gradient updates (the forward pass length).
- k_2: The maximum number of time steps backpropagated (the truncation window).

This approach allows the model to learn from recent history while mitigating the vanishing gradient problem, as we only backpropagate through a limited number of time steps, thus. It also reduces the computational cost, allowing for more efficient training of RNNs on long sequences.

This approach is particularly effective for tasks where long-term dependencies are less critical, as in fact it fails to capture dependencies that span more than k_2 time steps, but it can still be useful for many applications where recent context is more relevant.

5.3 Long Short-Term Memory (LSTM)

To address the limitations of Vanilla RNNs—specifically the **vanishing gradient problem**—**Long Short-Term Memory** (LSTM) networks were introduced. LSTMs utilize a gated architecture designed to maintain a persistent state over long sequences, allowing the model to learn when to forget old information and when to update with new inputs.

5.3.1 The Memory Cell and Cell State

The core innovation of the LSTM is the **cell state** (denoted as C_t), which acts as a long-term memory conveyor belt. Unlike the hidden state h_t, which undergoes multiple non-linear transformations at every time step, the cell state C_t interacts with the rest of the network primarly through linear operations.

This linear path allows gradients to flow through many time steps with minimal decay, effectively creating a "highway" for gradients during backpropagation, which is crucial for learning long-term dependencies in sequential data.

5.3.2 Gating Mechanisms

LSTMs regulate the cell state using three specialized "gates." Each gate consists of a sigmoid layer (σ) and an element-wise multiplication operation ($\odot$). These gates are learned parameters that determine the flow of information.

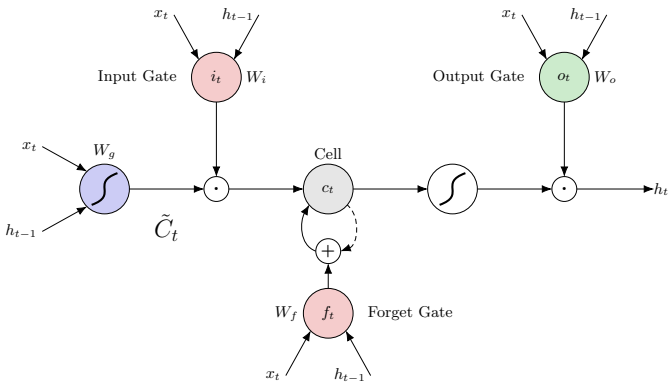


Figure 31: Internal gated architecture of an LSTM cell.

Input Gate (i_t): The input gate decides which values in the cell state will be updated. Simultaneously, a *candidate* cell state $\tilde{C}_t$ is created using a tanh layer to represent new information that could be added to the state.

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right) \tag{5.7}$$

The input gate takes the previous hidden state h_{t-1} and the current input x_t as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the cell state, where 0 means "do not update" and 1 means "update".

On the other hand, the candidate cell state $\tilde{C}_t$, similarly to the hidden state in a Vanilla RNN, is computed as a non-linear transformation of the current input and the previous hidden state:

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right) \tag{5.8}$$

Forget Gate (f_t): This gate decides what information from the previous cell state C_{t-1} is no longer useful and should be discarded.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{5.9}$$

It takes the previous hidden state h_{t-1} and the current input x_t as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the cell state. The output can be interpreted as a soft mask, where 0 means "completely forget," while 1 means "completely retain."

Output Gate (o_t): This gate decides what information to output from the cell state.

$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right) \tag{5.10}$$

It takes the previous hidden state h_{t-1} and the current input x_t as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state. The output can be interpreted as a soft mask, where 0 means "completely forget," while 1 means "completely retain."

5.3.3 Memory Cell Update

The new cell state C_t is updated by combining the gated previous state and the gated candidate state. This is the "update" step where the network actually forgets and learns:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{5.11}$$

By using the forget gate f_t, the network can reset the memory of a specific unit if it is no longer relevant, preventing the internal state from growing indefinitely.

Using the input gate i_t and the candidate cell state $\tilde{C}_t$, the network can add new information to the cell state, allowing it to learn from new inputs while retaining important past information.

5.3.4 Hidden State Generation

Finally, the hidden state h_t is updated. We pass the cell state through a tanh (to push the values between -1 and 1) and multiply it by the output gate o_t:

$$h_t = o_t \odot \tanh(C_t) \tag{5.12}$$

While C_t stores long-term information, h_t serves as the "working memory" used for the immediate prediction and as an input for the next time step.

5.3.5 Gradients and Backpropagation

The architectural beauty of the LSTM lies in its additive update rule for C_t. Even tough some non-linearities are present in the gates, the cell state C_t itself is updated through a combination of linear operations (multiplication and addition), which allows gradients to flow more easily during backpropagation. This effectively bypasses the vanishing gradient problem inherent in the multiplicative chain of standard RNNs.

5.4 Gated Recurrent Unit (GRU)

Gated Recurrent Units (GRUs) are a simplified version of LSTMs that combine the forget and input gates into a single "update gate" and merge the cell state and hidden state into a single state. This results in a more streamlined architecture that is computationally more efficient while still capturing long-term dependencies effectively.

5.4.1 GRU Cell Architecture

In a GRU cell, we have only two gates: the **update gate** (z_t) and the **reset gate** (r_t).

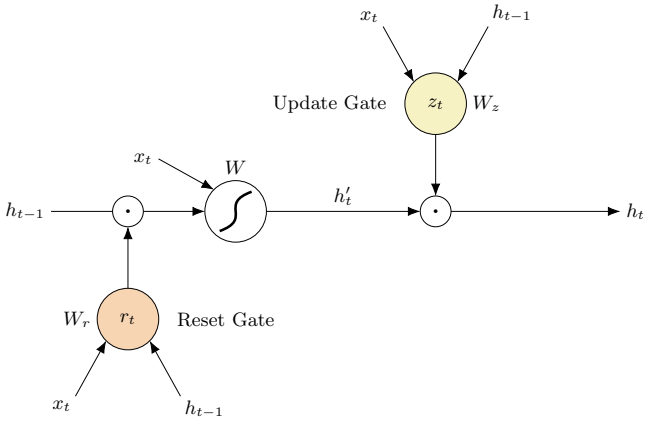


Figure 32: Internal gated architecture of a GRU cell.

Reset Gate (r_t): The reset gate determines how much of the previous hidden state h_{t-1} should be used when computing the candidate hidden state $\tilde{h}_t$.

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t] + b_r\right) \tag{5.13}$$

It takes the previous hidden state h_{t-1} and the current input x_t as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state, where 0 means "completely reset" and 1 means "completely retain."

Candidate Hidden State ($\tilde{h}_t$): The candidate hidden state is computed based on the reset gate and the previous hidden state.

$$\tilde{h}_t = \tanh\left(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h\right) \tag{5.14}$$

The reset gate r_t controls how much of the previous hidden state h_{t-1} is used in the computation of the candidate hidden state $\tilde{h}_t$. If r_t is close to 0, the previous hidden state is effectively ignored, allowing the model to reset its memory when necessary.

Update Gate (z_t): The update gate determines how much of the previous hidden state h_{t-1} and the candidate hidden state $\tilde{h}_t$ should be combined to form the new hidden state h_t.

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t] + b_z\right) \tag{5.15}$$

It takes the previous hidden state h_{t-1} and the current input x_t as input, applies a linear transformation followed by a sigmoid activation function, and outputs a value between 0 and 1 for each element in the hidden state, where 0 means "completely update" and 1 means "completely retain."

The new hidden state h_t is then computed as a linear interpolation between the previous hidden state h_{t-1} and the candidate hidden state $\tilde{h}_t$, weighted by the update gate z_t:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{5.16}$$

This allows the GRU to adaptively capture long-term dependencies while maintaining a simpler architecture compared to LSTMs.

6 Sequence Models

After the RNN breakthrough, it became clear that all the proposed architectures were unable to effectively capture long-term dependencies.

To address these issues, the concept of *attention* was introduced in sequence models, even before the emergence of the Transformer architecture. The attention mechanism allows the model to focus on specific parts of the input sequence when making predictions, enabling it to capture long-term dependencies more effectively.

In this section, we will explore how the introduction of attention shaped developments in the fields of Image Captioning and Machine Translation.

6.1 Image Captioning

Image captioning is the task of generating a textual description for a given image. This task combines **computer vision** for the image understanding and **natural language processing** for generating the caption. The main challenge in image captioning is to effectively capture the relevant features of the image and generate a coherent and contextually appropriate caption.

The usual approach to image captioning involves using a convolutional neural network (CNN) to extract features from the image, and then using a recurrent neural network (RNN) to generate the caption based on those features.

6.1.1 No Attention - Show and Tell

Show and Tell is one of the early models for image captioning. Following the encoder-decoder architecture proposed for machine translation, Show and Tell uses a CNN as the encoder to extract features from the image and an RNN as the decoder to generate the caption.

The architecture of Show and Tell consists of two main components:

- **Encoder**: A CNN (such as Inception or ResNet) is used to extract a fixed-length feature vector from the input image. This feature vector captures the high-level visual information of the image.
- **Decoder**: An RNN (such as LSTM) is used to generate the caption word by word. The RNN takes the feature vector from the encoder as input and generates the caption sequentially, predicting one word at a time until it generates an end token.

At the first time step, the RNN receives as input the **feature vector** from the encoder and a special start token. At each subsequent time step t, the RNN

receives as input the word generated at time step $t-1$ and the hidden state from the previous time step. The RNN continues to generate words until it produces an end token, indicating the end of the caption.

Inference At inference time, two different strategies can be used to generate captions:

- **Sampling**: At each time step, the model samples a word from the predicted probability distribution. While this can lead to more diverse and creative captions, it also increases the risk of generating grammatically incorrect or nonsensical sequences.
- **Beam Search**: Instead of sampling, beam search keeps track of the top k most likely sequences at each time step, where k is the beam width. At each step, the model expands each of the k sequences by one word and keeps only the top k resulting sequences based on their cumulative probabilities. Beam search is more computationally expensive than sampling but often results in more coherent and higher-quality captions.

Evaluation Metrics In order to evaluate the performance of image captioning models, we must be able to compare the generated captions with human-generated reference captions. To address this, the BLUE (Bilingual Evaluation Understudy) metric was introduced.

BLUE is a precision-based metric that measures the overlap between the generated caption and one or more reference captions. It calculates the **n-gram precision**, which is the proportion of n-grams (contiguous sequences of n words) in the generated caption that are also present in the reference captions.

Limitations Despite its success, this model faced several inherent limitations:

- **Information Bottleneck**: The fixed-length feature vector may not capture all relevant details, especially in complex images with multiple interacting objects.
- **Vanishing Visual Context**: Because the RNN only "sees" the image at the first time step, the visual influence can weaken as the sentence grows, leading the model to rely more on language probability than the actual image.
- **Static Features**: The feature vector is global and static; the model cannot "shift its gaze" to different parts of the image as it describes different objects.

6.1.2 Attention - Show, Attend and Tell

To overcome the limitations of the Show and Tell model, the Show, Attend and Tell model was proposed, introducing an attention mechanism to allow the model to focus on different parts of the image while generating the caption.

The architecture of Show, Attend and Tell consists of three main components:

- **Encoder**: Similar to Show and Tell, a CNN is used to extract features from the image. However, instead of producing a single fixed-length feature vector using a fully connected layer, the encoder produces a set of feature vectors corresponding to different regions of the image, maintaining spatial information about the image. This is done by extracting features from the last convolutional layer of the CNN, avoiding the fully connected layers that would otherwise collapse the spatial dimensions.

  In particular, the encoder produces a set of feature vectors $\{a_1, a_2, ..., a_L\}$, where L is the number of regions in the image (e.g., 14x14 = 196 regions) and each a_i is a feature vector corresponding to a specific region of the image.

- **Attention Mechanism**: The attention mechanism f_{att} allows the model to dynamically focus on different parts of the image while generating the caption. The dynamic attention weights are computed at each time step, as a function of the **previous hidden state**.

  Once the attention weights are computed, they are used to compute a weighted sum of the feature vectors from the encoder, resulting in a **context vector** that captures the relevant visual information for generating the next word in the caption.

- **Decoder**: The LSTM decoder is responsible for generating the caption word by word. It takes as input the context vector z_t computed from the attention mechanism, the previous hidden state h_{t-1}, and the previously generated word y_{t-1}. The decoder generates the next word in the caption based on this information, allowing it to produce more accurate and contextually relevant captions by focusing on different parts of the image at each time step.

Attention Types Two types of attention mechanisms are commonly used in image captioning:

- **Soft Attention**: This is a differentiable attention mechanism that computes a weighted average of the image features based on the attention weights. The model can be trained end-to-end using backpropagation, as the attention weights are differentiable with respect to the model parameters.

- **Hard Attention**: This is a non-differentiable attention mechanism that selects a single region of the image to focus on at each time step. Since the selection process is non-differentiable, reinforcement learning techniques (such as REINFORCE) are used to train the model, treating the attention selection as a stochastic policy.

6.1.3 Machine Translation

7 CLIP

CLIP (Contrastive Language-Image Pretraining) is a SOTA *Vision-Language-Model (VLM)*, designed to learn a joint embedding space for images and text, enabling it to perform various tasks such as image classification, object detection, and zero-shot learning.

CLIP is composed of two main components:

- An **image encoder**, typically a convolutional neural network (CNN) or a vision transformer (ViT), which processes images and produces feature representations.
- A **text encoder**, typically a transformer-based architecture, which processes text and produces feature representations.

The key innovation of CLIP is its use of a **contrastive learning approach** to train both encoders simultaneously. This approach allows CLIP to learn a shared embedding space where semantically similar images and text are close together, while dissimilar pairs are far apart.

Before CLIP Before CLIP, the traditional learning approach for computer vision involved creating a training dataset of pairs of images and their corresponding labels. This approach had a few limitations:

- It required a large amount of labeled data, which can be expensive and time-consuming to collect.
- It was limited to the specific tasks for which it was trained, and could not easily generalize to new tasks or domains.
- To introduce new classes, the model had to be retrained on a new dataset, which was not efficient.

Since CLIP was trained on a large dataset of image-text pairs, it can perform zero-shot learning, meaning it can recognize and classify images based on textual descriptions without needing to be retrained on specific classes.

7.1 Ideas that lead to CLIP

In reality, CLIP didn't introduce anything revolutionary, but rather correctly combined existing ideas and innovations in the field of machine learning and computer vision. Some of the key ideas that led to the development of CLIP include:

- **Webly supervised learning**: The idea of using large amounts of data from the web to train models, which has been a common practice in machine learning for many years.
- **Weakly supervised learning**: Using talked video data, which contains both visual and textual information, to learn a joint embedding space for images and text.
- **Going at scale**: The idea of training large models on massive datasets was already exploited in the field of Natural Language Processing (NLP) with models like GPT-3. CLIP applied this idea to the field of computer vision, demonstrating that scaling up the model and dataset can lead to significant improvements in performance.

7.2 Dataset

Previous works on this field had used datasets of limited size, such as MS COCO, which contains only 100K. A new dataset was needed to assess wether the proposed approach could scale up.

The dataset used to train CLIP, called *WebImageText*, consists of 400 million image-text pairs collected from the internet. It was created by scraping the web with more than 500k queries, built using a combination of common words, combinations of words, and random words. The queries were designed to be diverse and cover a wide range of topics and concepts, ensuring that the dataset is representative of the variety of images and text found on the internet. To have an approximative class balance, each query includes up to 20k image-text pairs.

This dataset is significantly larger than previous datasets used for training vision-language models, but it is also noisier, as the data was minimally curated and quality of the image-text pairs is not guaranteed. This causes that the model reflects the biases and limitations of the data.

7.3 Contrastive Pretraining

As mentioned before, CLIP uses a contrastive learning approach to train both the image and text encoders simultaneously. The training objective is to **maximize** the cosine similarity between the embeddings of the N matching image-text pairs in a batch, while **minimizing** the cosine similarity of the $N^2 - N$ incorrect pairs.

The loss function used for training CLIP is a softmax-based contrastive loss,

defined as:

$$\mathcal{L} = -\frac{1}{2N}\sum_{i=1}^{N}\left(\underbrace{\log\frac{\exp(\text{sim}(v_i,t_i)/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(v_i,t_j)/\tau)}}_{Image\rightarrow Text} + \underbrace{\log\frac{\exp(\text{sim}(t_i,v_i)/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(t_i,v_j)/\tau)}}_{Text\rightarrow Image}\right), \tag{7.1}$$

where v_i and t_i are the embeddings of the i-th image and text pair, sim is the cosine similarity function, and τ is a temperature hyperparameter that controls the sharpness of the distribution. This loss encourages the model to learn a shared embedding space where semantically similar images and text are close together, while dissimilar pairs are far apart.

Temperature : The temperature τ is a hyperparameter that controls the sharpness of the distribution in the contrastive loss. A lower temperature makes the distribution sharper, which can lead to better performance but also increases the risk of overfitting. Conversely, a higher temperature makes the distribution softer over the possible classes, which can help to prevent overfitting but may lead to worse performance.

Training Phase : Since the WebImageText dataset is very large, both the image and text encoders are trained from scratch, without using any pretraining on other datasets. Simple data augmentation techniques are applied to the images.

Limitations of the training approach This training approach has some limitations regarding the computational resources required. In particular, since we need to compute the cosine similarity between all pairs of images and text in a batch (N^2 pairs), the size of the batch is limited by the available memory.

7.4 Inference

During inference, CLIP can be used for various tasks such as image classification, object detection, and zero-shot learning.

- **Image classification**: Given an image, CLIP can be used to classify it by comparing its embedding to the embeddings of a set of candidate labels (text descriptions);
- **Text to Image retrieval**: Given a text query, CLIP can be used to retrieve relevant images by comparing the text embedding to the embeddings of a set of candidate images. First, the text query is encoded using

the text encoder to obtain its embedding. Then, the cosine similarity between the text embedding and the embeddings of a set of candidate images is computed. The images with the highest similarity scores are returned as the most relevant results.

- **Image to Image retrieval**: Given an image query, CLIP can be used to retrieve relevant images by comparing the image embedding to the embeddings of a set of candidate images. Similarly to text to image retrieval, the image query is encoded using the image encoder to obtain its embedding. Then, the cosine similarity between the image embedding and the embeddings of a set of candidate images is computed. The images with the highest similarity scores are returned as the most relevant results.
- **Zero-shot learning**: CLIP enables zero-shot learning, meaning it can recognize and classify images based on textual descriptions without needing to be retrained on specific classes.

7.5 Fine Tuning

Once the power of CLIP was demonstrated on zero-shot learning tasks, the next step was to explore how to further improve its performance on specific tasks. A first approach was to fine-tune either the image encoder, the text encoder, or both on a specific dataset. This was found to be ineffective, as the model had already learned a powerful shared embedding space, and fine-tuning on a specific dataset often led to overfitting and a decrease in performance on other tasks.

Other approaches based on prompt engineering were found to be more effective in improving the performance of CLIP on specific tasks, without the need for fine-tuning the model itself. Here, we will discuss some of the most effective techniques for improving CLIP's performance through prompt engineering.

7.5.1 Basic Prompt Engineering

: Prompt engineering is the process of designing and optimizing the input prompts to improve a model's performance on specific tasks. Since CLIP learns a shared embedding space, the phrasing of the text prompts significantly impacts the results.

For example, since images are rarely described with a single word in the wild, using a template like `"a photo of a [class]"` often outperforms using the raw class name alone. On ImageNet, this simple modification yielded a gain of 1.3% in accuracy.

7.5.2 Prompt Ensembling

Further improvements can be achieved by using multiple prompts for each class and averaging their embeddings. For the class "dog," one might ensemble prompts like *"a photo of a dog," "a picture of a dog,"* and *"a photo of a cute dog."* On ImageNet, this approach provided an additional 3.5% gain in accuracy compared to a single prompt.

7.5.3 Advanced Approaches

Prompt engineering can still be difficult and time-consuming. Many times a slight change in the prompt can lead to a significant change in performance, with no clear way to predict which prompts will work best for a given task.

To address this, more advanced approaches have been proposed.

CoOp CoOp (Context Optimization) is a method that learns continuous **prompt vectors** instead of using discrete text prompts. These prompt vectors are then fed into the text encoder to generate the class embeddings. Different choices can be made, such as the number of prompt vectors, their dimensionality, where they are inserted in the text encoder (e.g., before the class name, after the class name, or both), and whether they are shared across classes or specific to each class.

CoOp optimizes the prompt vectors using a small amount of labeled data from the target task, allowing it to adapt the prompts to better suit the specific task at hand. This approach has been shown to significantly improve the performance of CLIP on various tasks, especially when the amount of labeled data is limited.

Even though the results are quite impressive, they are still not humanly interpretable, as the learned prompt vectors may not correspond to any meaningful words or phrases. Also, the approach was shown to have poor generalization performance to new classes, as the learned prompt vectors may be overfitted to the specific classes seen during training.

CoCoOp CoCoOp (Contextualized Context Optimization) is an extension of CoOp that addresses the issue of generalization to new classes.

CoCoOp learns a **meta-network** that generates some meta-prompts based on the output of the image encoder. These meta-tokens are them summed with the learned prompt vectors to generate the final prompts that are fed into the text encoder. This allows CoCoOp to adapt the prompts based on the specific image being processed, improving generalization to new classes and reducing overfitting to the classes seen during training.

The problem with this approach is that it is more computationally expensive than CoOp, as it requires an additional forward pass through the meta-network for each image, during the training phase.

Visual Prompt Tuning All the approaches described so far have focused on optimizing the text prompts, while keeping the image encoder fixed. Visual Prompt Tuning is a method that optimizes the input to the image encoder instead of the text encoder. Similarly to CoOp, it learns a set of continuous prompt vectors that are instead added to the input image before being fed into the image encoder.

Multimodal prompt Learning Sometimes, optimizing only the text prompts or only the image prompts leads to suboptimal performance, as ther's no general rule to determine which one is more effective for a given task. Multimodal Prompt Learning is a method that optimizes both the text and image prompts simultaneously, allowing for a more holistic approach to prompt engineering.

Using a small neatwork (for example, a transformer), the method learns to generate both text and image prompts based on the input image and the target task. The prompts are then fed into the respective encoders to generate the final embeddings, which are used for classification or other downstream tasks.

7.6 Beyond CLIP - SigLIP

To resolve the limitations of CLIP, presented in Equation 7.3, a new method called SigLIP (Sigmoid Loss for Language-Image Pretraining) was proposed. Instead of using a softmax-based contrastive loss, where each pair of image and text is compared to all other pairs in the batch, SigLIP uses a sigmoid-based loss that compares each pair independently, treating the problem as a binary classification.

For each pair of image and text, the model predicts a score that indicates whether the pair is a match or not (e.g., whether the image and text are semantically related). Because each pair is treated independently, the batch size can be much larger without running into memory issues, allowing for more efficient training on large datasets.